MACHINE LEARNING

Neural networks and language models from first principles

UNIT 4

Making It Fast

The machine, profiling, numerics, kernels, parallel training, an inference engine, quantisation and serving

4.1 The machine

4.2 Profiling and memory

4.3 Numerics

4.4 Kernels

4.5 Parallel training

4.6 An inference engine in C++

4.7 Quantisation

4.8 Serving

First edition, October 2026

Contents

About this unit 3
- Before you start 3
- What this unit needs 3

4.1 The machine 4
- Core ideas 4
- Reading 7
- Labs 7
- Problem set 8

4.2 Profiling and memory 10
- Core ideas 10
- Reading 13
- Labs 13
- Problem set 14

4.3 Numerics 16
- Core ideas 16
- Reading 19
- Labs 20
- Problem set 21

4.4 Kernels 22
- Core ideas 22
- Reading 31
- Labs 31
- Problem set 32

4.5 Parallel training 34
- Core ideas 34
- Reading 39
- Labs 39
- Problem set 40

4.6 An inference engine in C++ 42
- Core ideas 42
- Reading 50
- Labs 51
- Problem set 52

4.7 Quantisation 53
- Core ideas 53
- Reading 57
- Labs 57
- Problem set 59

4.8 Serving 60
- Core ideas 60
- Reading 65
- Labs 66
- Problem set 67

Sign-off 68

Machine Learning, First edition, October 2026. © 2026 O'Neill Group LLC. PyTorch is a trademark of the Linux Foundation. Apple, Apple Silicon and Metal are trademarks of Apple Inc. NVIDIA and CUDA are trademarks of NVIDIA Corporation. Vulkan is a registered trademark of the Khronos Group Inc.

Hugging Face is a trademark of Hugging Face, Inc. All other trademarks belong to their owners. The models and datasets the course uses keep their own licences, listed in the Handbook.

UNIT 4

About this unit

Unit 4 is about speed and size: what limits a model's training and inference, and how those limits are raised. It measures the machine's roofline and places every operation of your transformer on it, profiles a training step and writes down its memory equation, and studies the number formats that make large models possible. It then writes the kernels behind the operations, in Metal or Vulkan, parallelises training across processes, builds an inference engine in C++, quantises models to 8 and 4 bits, and ends with a server that batches requests continuously, pages its KV cache and decodes speculatively.

Chapter	You build	Hours
4.1	Your machine's roofline, measured, with every transformer operation placed on it	5
4.2	A training step profiled, its memory predicted and measured, and checkpointing	6
4.3	Floating-point formats, overflow and underflow, loss scaling and fp8	5
4.4	Matrix multiplication, softmax and FlashAttention as GPU kernels, in Metal or Vulkan	9
4.5	A ring all-reduce, data parallelism, ZeRO, tensor and pipeline parallelism	7
4.6	An inference engine in C++ with a tokeniser, threads, SIMD and a KV cache	8
4.7	Int8 and int4 weights, GPTQ, a quantised KV cache, and the engine's quantised paths	6
4.8	Continuous batching, a paged KV cache, speculative decoding and a streaming server	6

Before you start

Finish Unit 2; Unit 3 helps but is not required. For Chapters 4.6 and 4.7, install a C++20 compiler, CMake and Ninja; for the Vulkan track of Chapter 4.4, the Vulkan SDK.

What this unit needs

Your own machine, which this unit measures. Chapter 4.4's Metal track needs an Apple Silicon Mac; its Vulkan track runs on any GPU with a Vulkan 1.3 driver, including Macs through MoltenVK.

CHAPTER 4.1

The machine

TIME	5 hours
YOU BUILD	Your GPU's memory bandwidth and arithmetic peak, measured; its roofline; every operation of a layer of Chapter 2.5′s model timed, its FLOPs and bytes counted, and placed against the roofline; and an account of each operation's distance from its bound.
NEEDS	Chapters 1.1 and 2.4.

WHY IT MATTERS

Unit 1 measured matrix multiplications and found them limited sometimes by arithmetic and sometimes by memory; Unit 2 trained a transformer at about half of the GPU's arithmetic peak. This unit is about closing such gaps, and each attempt starts by finding what limits the operation. The *roofline model* finds it from four numbers, the machine's peak arithmetic and memory bandwidth and the operation's FLOPs and bytes. This chapter measures all four for the machine on your desk and the model you trained.

SKIP TEST

1. State the roofline: the attainable FLOP/s of an operation with arithmetic intensity I on a machine with peak P and bandwidth B.
2. What is the arithmetic intensity of multiplying an $N \times d$ matrix by a $d \times d$ one in float32, and of an element-wise addition?
3. Why can a GPU with ten times a CPU's arithmetic peak run an RMS normalisation only slightly faster?
4. What is the ridge point of a machine, and how do you compute it?

If all four are easy, read the section on the transformer's operations and do Labs 4.1.2 and 4.1.4.

Core ideas

Two numbers for a machine

A GPU is thousands of simple arithmetic units fed by a memory system. Its *peak* is how many FLOPs per second the units can perform, and its *bandwidth* is how many bytes per second memory can deliver to them. Between the units and main memory sit faster, smaller stores: registers private to each thread, a small *shared* or *threadgroup* memory per group of threads, and caches. Chapter 4.4 programs them directly. In this chapter they explain why measured numbers fall short of datasheets.

The M2 Pro's CPU and GPU share one pool of memory, Apple's *unified memory*, with a published bandwidth of 200 GB/s, and its GPU's 19 cores have a peak of about 6.8 TFLOP/s in float32. A data-centre GPU such as NVIDIA's H100 has 3.35 TB/s of bandwidth and nearly 1,000 TFLOP/s for bfloat16 matrix multiplications on its tensor cores: about seventeen times the bandwidth and well over a hundred times the matrix arithmetic. The ratio of the two numbers matters most for what follows, and it is larger on the H100.

Measuring bandwidth with a copy, and the arithmetic peak with a large product — code/u4/roofline.py

```
a = torch.empty(64 * 2**20, device=dev)
b = torch.empty_like(a)
seconds = median_time(lambda: b.copy_(a), runs=20, dev=dev)
bandwidth = 2 * a.numel() * 4 / seconds / 1e9
x, y = torch.randn(4096, 4096, device=dev), torch.randn(4096, 4096, device=dev)
peak = 2 * 4096**3 / median_time(lambda: x @ y, runs=20, dev=dev) / 1e9
```

The M2 Pro's GPU, measured

```
$ uv run -m u4.roofline
mps: copying 256 MiB runs at 175 GB/s; a 4096 x 4096 product at 5,528 GFLOP/s
ridge point: 32 FLOP per byte
```

The roofline

An operation that performs F FLOPs and moves M bytes has an *arithmetic intensity* $I = F/M$. It cannot run faster than the arithmetic allows, P, nor faster than memory can feed it, IB:

$$\text{attainable FLOP/s} = \min(P, IB).$$

Plotted against intensity on logarithmic axes, the bound is a sloping line that meets a flat roof at the *ridge point* $I = P/B$. An operation left of the ridge is *memory-bound*: faster arithmetic does not help it, and only moving fewer bytes does. An operation right of it is *compute-bound*, and only less or faster arithmetic helps.

A transformer layer on the roofline

The program builds tensors with the shapes of one layer of Chapter 2.5's model at its training batch, 32 sequences of 512 tokens, and times each operation of a layer's forward pass, with its FLOPs and the minimum bytes it must move:

Each operation with its FLOPs and minimum bytes — code/u4/roofline.py

```
ops = {
    "q, k, v projection": (
        lambda: h @ w_qkv,
        2 * N * d * w_qkv.shape[1],
        4 * (N * d + w_qkv.numel() + N * w_qkv.shape[1]),
    ),
    "attention (SDPA)": (
        lambda: F.scaled_dot_product_attention(q, k, v, is_causal=True),
        2 * B * H * T * T * hd,
        4 * 4 * B * H * T * hd,
    ),
    "output projection": (lambda: h @ w_o, 2 * N * d * d, 4 * (2 * N * d + d * d)),
    "gate and up": (
        lambda: h @ w_up,
        2 * N * d * 2 * ff,
        4 * (N * d + d * 2 * ff + N * 2 * ff),
    ),
    "SiLU(gate) * up": (lambda: F.silu(gate) * up, 6 * N * ff, 4 * 3 * N * ff),
    "down projection": (
        lambda: gate @ w_down,
        2 * N * ff * d,
        4 * (N * ff + ff * d + N * d),
    ),
    "RMSNorm": (lambda: F.rms_norm(h, (d,), weight), 4 * N * d, 4 * 2 * N * d),
    "residual add": (lambda: h + h, N * d, 4 * 3 * N * d),
    "logits": (lambda: h @ w_head, 2 * N * d * V, 4 * (N * d + d * V + N * V)),
    "cross-entropy": (lambda: F.cross_entropy(logits, targets), 5 * N * V, 4 * N * V),
}
```

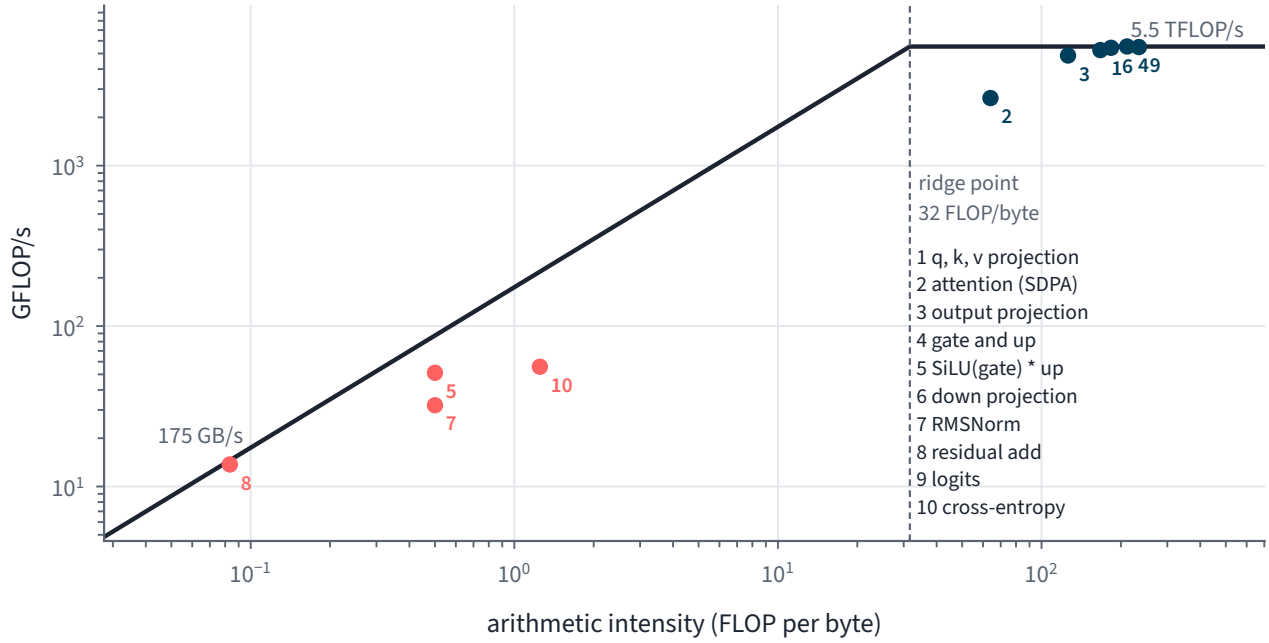


Figure 4.1.1. The M2 Pro's measured roofline, with the operations of a transformer layer at their measured speeds.

One layer's operations against the roofline

```
one layer of the Chapter 2.5 model, batch 32 x 512 tokens, float32:
operation              GFLOP      MB  FLOP/B      ms  GFLOP/s   GB/s   bound
q, k, v projection     17.18     103   167.2    3.27     5253     31   compute,  95% of it
attention (SDPA)        8.59     134    64.0    3.26     2638     41   compute,  48% of it
output projection       8.59      68   126.0    1.77     4860     39   compute,  88% of it
gate and up            47.24     224   211.0    8.54     5532     26   compute, 100% of it
SiLU(gate) * up         0.14     277     0.5    2.71       51    102   memory,  59% of it
down projection        23.62     129   183.5    4.35     5428     30   compute,  98% of it
RMSNorm                 0.03      67     0.5    1.05       32     64   memory,  37% of it
residual add            0.01     101     0.1    0.61       14    165   memory,  94% of it
logits                137.44     587   234.1   25.06     5485     23   compute,  99% of it
cross-entropy           0.67     537     1.2   12.03       56     45   memory,  26% of it
```

The measured machine falls short of its datasheet. Copies run at 175 GB/s of the published 200, and a large product at 5.5 of the nominal 6.8 TFLOP/s, which puts the ridge at 32 FLOP per byte. The matrix multiplications run at 88% to 100% of that peak, so they are compute-bound and PyTorch's kernels leave little to gain. Attention reaches about half the peak, and Chapter 4.4 writes such a kernel to see why. The element-wise operations sit far left of the ridge, where only bandwidth matters. The residual addition streams at 94% of the copy's bandwidth, but SiLU-gating and RMS normalisation reach 59% and 37% of their bounds. Counting each layer's two norms and two additions, the element-wise operations take about a fifth of the layer's time for well under 1% of its FLOPs. The cross-entropy is the worst, at a quarter of its bound. PyTorch computes a log-softmax over all 537 MB of logits and writes it out before picking each target's entry, so it moves several times the bytes a fused kernel would need.

What the roofline says about everything that follows

The roofline sorts this unit's techniques by the bound they attack. Fusing element-wise operations, so that a normalisation, a SiLU and a multiplication read and write memory once instead of three times, raises their effective intensity; Chapter 4.2 measures what `torch.compile` gains that way. FlashAttention, in Chapter 4.4, avoids writing the attention scores to memory at all. Generating text one token at a time turns every matrix multiplication into a matrix-vector product with intensity near 0.5, far left of the ridge. Quantisation, in Chapter 4.7, raises it by moving fewer bytes per weight, and batching, in Chapter 4.8, by reusing each weight for many sequences. Data-centre GPUs, whose ridge points lie further right than the M2 Pro's, make all of these matter more.

KEY IDEA

Each operation is bound by arithmetic or by memory, depending on its intensity. Measure the machine's two peaks and count each operation's FLOPs and bytes, and the roofline gives how fast the operation could run and what would make it faster.

IN PYTORCH

PyTorch's operations each read their inputs from memory and write their outputs back, so a chain of element-wise operations is a chain of memory-bound kernels. `torch.compile` fuses such chains, and `torch.utils.flop_counter.FlopCounterMode` counts an operation's FLOPs for you. Vendor tools, Nsight Compute for NVIDIA and Xcode's Metal profiler for Apple, report measured bytes and FLOPs per kernel and draw rooflines of their own.

Reading

- Samuel Williams, Andrew Waterman and David Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures", *Communications of the ACM* 52(4), 2009.
- Horace He, "Making Deep Learning Go Brrrr From First Principles", 2022: compute, memory and overhead bounds in PyTorch.
- NVIDIA, *H100 Tensor Core GPU Architecture* white paper, 2022, and Apple's documentation of the M2 family's GPU.

Labs

LAB 4.1.1 Bandwidth and the caches — 1 hour

Goal. See the memory hierarchy in a bandwidth curve.

Code. `code/u4/roofline.py`

1. Measure copy bandwidth on your GPU for sizes from 64 KiB to 1 GiB, doubling each time, with enough repetitions for each to take at least 10 ms.
2. Plot bandwidth against size and mark where the curve changes.

DONE WHEN

- [ ] Your plot covers at least 15 sizes.
- [ ] You can relate each step in the curve to a level of the memory hierarchy, or explain why there is none.

EVIDENCE FOR THE NOTEBOOK

- The plot and its interpretation.

LAB 4.1.2 The shape of a product — 1.5 hours

Goal. Map a matrix multiplication's speed against its shape.

Code. `code/u4/roofline.py`

1. Time products of an $N \times 512$ matrix by a 512×512 one for N from 1 to 16,384, and compute each one's intensity.
2. Place them on your roofline, and find the N at which the product becomes compute-bound.

DONE WHEN

- [ ] Your points follow the roofline's shape, and you can name the crossover N.

- [ ] You can relate $N = 1$ to generating one token and the largest N to training.

EVIDENCE FOR THE NOTEBOOK
- The table and the plot.

LAB 4.1.3 Fusing element-wise operations — 1 hour

Goal. Measure what fusing element-wise operations gains.

Code. `code/u4/roofline.py`

1. Time RMSNorm followed by SiLU-gating as separate PyTorch operations, and the same function compiled with `torch.compile`.
2. Count the bytes each version must move at least, and compare the measured speed-up with the ratio of bytes.

DONE WHEN
- [ ] Your two byte counts and two times are recorded.
- [ ] The speed-up is within a factor of two of the byte ratio, or you can explain why not.

EVIDENCE FOR THE NOTEBOOK
- The counts, the times and the comparison.

LAB 4.1.4 Another machine on paper — 1.5 hours

Goal. Predict the same layer on a data-centre GPU.

Code. `code/u4/roofline.py`

1. Using an H100's published bandwidth and bfloat16 peak, predict the time of each operation in the table, halving the bytes for bfloat16.
2. Find the operations whose share of the layer's time would grow most compared with the M2 Pro.

DONE WHEN
- [ ] Your predictions cover every operation.
- [ ] You can say which operations would dominate on the H100 and what that implies for optimising them.

EVIDENCE FOR THE NOTEBOOK
- The predictions and the conclusion.

Problem set

1. Compute the ridge points of the M2 Pro from your measurements and of an H100 for bfloat16. What do they imply about the operations each machine runs well?
2. Show that multiplying an $N \times d$ matrix by a $d \times d$ matrix in float32 has intensity $Nd/(2(2N + d))$, counting the bytes of both inputs and the output. What is it for $N = 1$ and $N = d$?
3. RMS normalisation of Nd values reads them, computes a mean square per row and writes the result. What is its intensity, and its best time on the M2 Pro for $N = 8192$, $d = 512$?
4. Why does the measured cross-entropy run far below its memory bound, and what would a fused implementation have to do?
5. The attention row of the table counts bytes as if the scores never touched memory. When is that true, and what would the bytes be otherwise?

COMPLETION CHECKLIST

- [ ] I can measure a machine's bandwidth and arithmetic peak, and draw its roofline.
- [ ] I can count an operation's FLOPs and minimum bytes and compute its intensity.
- [ ] I can place a transformer's operations on the roofline and say what limits each.
- [ ] I can predict which optimisations will help an operation from its position on the roofline.
- [ ] Lab 4.1.1–4.1.4 done-when criteria all hold, with evidence filed.

CHAPTER 4.2

Profiling and memory

TIME	6 hours
YOU BUILD	A training step of Chapter 2.5's model split into its forward pass, backward pass and optimiser step, and its forward pass split by module; the memory equation for training, with the weights and optimiser state predicted exactly and the activations measured per token; activation checkpointing, measured; and an account of what `torch.compile` gains by fusion.
NEEDS	Chapters 2.5 and 4.1.

WHY IT MATTERS

Before changing a training run, measure where its time goes, which is rarely where intuition puts it, and where its memory goes, which decides what fits at all. This chapter measures both for the model you trained, writes down the memory equation that every large training run is planned with, and examines the two standard trades of time for memory and of memory traffic for time: activation checkpointing and fusion.

SKIP TEST

1. Why must GPU work be synchronised before stopping a timer, and what happens if it is not?
2. How many bytes per parameter does training with AdamW in float32 need before activations, and what are they?
3. What determines the memory of activations, and how does it grow with batch size and sequence length?
4. What does activation checkpointing trade, and roughly how much time does it cost?

If all four are easy, read the section on activations and do Labs 4.2.2 and 4.2.4.

Core ideas

Timing a training step

A GPU runs work asynchronously: each PyTorch call queues a kernel and returns at once, and the CPU runs ahead until it must wait for a result. A timer stopped without waiting measures only the queueing. Synchronising after each phase of a step separates the forward pass, the backward pass and the optimiser step:

One training step of the Chapter 2.5 model in eager mode, by phase

```
$ uv run -m u4.profile
one training step of the Chapter 2.5 model, 16 x 512 tokens, eager mode: 842 ms
  forward     241.1 ms  28.6%
  backward    590.6 ms  70.1%
  optimiser    10.4 ms   1.2%
```

One step at 16 sequences takes 842 ms in eager mode. The backward pass takes 2.4 times as long as the forward pass, a little more than the factor of two that its FLOPs predict, since more of its work is in operations less efficient than the forward pass's matrix multiplications. The optimiser's step, which touches each parameter a few times, costs about 1%.

Splitting the forward pass by module needs a timer around each one. Hooks, functions that PyTorch calls before and after a module runs, provide that without changing the model. Each hook synchronises, which adds overhead and stops the GPU overlapping work, so the shares are approximate.

Timing modules with forward hooks — code/u4/profile.py

```
def module_times(model, x, dev, runs: int = 5) -> dict[str, float]:
    times, starts = {}, {}

    def before(name):
        def hook(module, args):
            device.sync(dev)
            starts[name] = time.perf_counter()

        return hook

    def after(name):
        def hook(module, args, out):
            device.sync(dev)
            times[name] = times.get(name, 0.0) + time.perf_counter() - starts[name]

        return hook

    handles = []
    for name, module in (
        ("embedding", model.embed),
        ("final norm", model.norm),
        ("logits", model.head),
    ):
        handles += [
            module.register_forward_pre_hook(before(name)),
            module.register_forward_hook(after(name)),
        ]
    for block in model.blocks:
        for name, module in (
            ("attention", block.attn),
            ("feed-forward", block.mlp),
            ("norms", block.attn_norm),
            ("norms", block.mlp_norm),
        ):
            handles += [
                module.register_forward_pre_hook(before(name)),
                module.register_forward_hook(after(name)),
            ]
    with torch.no_grad():
        for _ in range(runs):
            model(x)
    for h in handles:
        h.remove()
    return {k: v / runs for k, v in times.items()}
```

The forward pass by module

```
forward time by module (synchronised after each, which adds overhead):
  feed-forward     62.7 ms  37.9%
  attention        57.7 ms  34.9%
  norms            30.5 ms  18.4%
  logits           12.3 ms   7.4%
  final norm        2.0 ms   1.2%
  embedding         0.4 ms   0.2%
```

The feed-forward networks hold three-fifths of the parameters and more than half of the forward pass's FLOPs, but take 38% of its time. Attention, with its scores and softmax, takes 35%, and the output layer, a seventh of the FLOPs in one large, efficient product, takes 7%. The norms, with almost no FLOPs, take 18%.

A full profiler records more. PyTorch's `torch.profiler` records every operation with its time and memory, and on Apple Silicon `torch.mps.profiler` emits signposts that Instruments shows on a timeline with the GPU's own counters.

Fusion

The norms take a far larger share of the time than of the FLOPs because they are memory-bound. Each reads and writes its tensor, and a chain of such operations (a norm, a rotation, a SiLU, a multiplication, a residual addition) reads and writes it again and again. `torch.compile` traces the model and generates kernels that perform such chains in one pass, keeping intermediate values in registers. Chapter 2.5 measured its effect on training: about 9,400 tokens per second in eager mode and 18,100 compiled, nearly double, with the matrix multiplications unchanged and everything else fused.

The memory equation

Training memory has four parts. The *weights* and their *gradients* take 4 bytes each per parameter in float32, and AdamW's two *moments* take 8, so together they need 16 bytes per parameter, fixed for a given model.

Weights, gradients and moments — code/u4/profile.py

```
params = gpt.param_count(c)
state = params * (4 + 4 + 8)
```

The fourth part, the *activations*, holds every intermediate tensor that the backward pass will need, and grows with the number of tokens in the batch. The program measures memory at rest and at the end of a forward pass for three batch sizes:

Memory at rest, and the activations of a forward pass

```
memory: 27.8M parameters x 16 bytes for weights, gradients and Adam's two moments = 424 MiB
    before any activations
 batch  resting MiB  activations MiB  per token KiB  step ms
     4          490            1005          502.7      217
     8          556            2008          502.1      426
    16          688            4017          502.1      845
```

The state at rest is the predicted 424 MiB plus buffers that the allocator keeps from earlier steps, which grow with the batch. The activations of a forward pass are 502 KiB per token at every batch size, so they grow in exact proportion to the tokens in the batch.

An estimate per token per layer counts what the backward pass needs: the block's input and its normalised version, the queries, keys and values, the attention output and its saved statistics, and in the feed-forward network the gate and up projections and their SiLU and product, each d_{ff} wide. For this model that is about $10d + 4d_{\text{ff}}$ numbers, or about 43 KB per token per layer in float32. Adding attention's saved weights, where the kernel keeps them, and the logits, which at 8,192 values per token are large, brings the total near the measured 500 KiB per token. Activations grow linearly with batch and sequence length, and for long sequences any attention implementation that stores its scores adds a term quadratic in length.

Activation checkpointing

Activations can be traded for time. *Activation checkpointing* keeps only each block's input during the forward pass and recomputes the block's internals during the backward pass, when they are needed. Memory falls to roughly one tensor per block, and time rises by about one extra forward pass, which is a third of a training step counted in FLOPs.

Wrapping each block so its internals are recomputed in the backward pass — code/u4/profile.py

```
class Checkpointed(torch.nn.Module):
    def __init__(self, block):
        super().__init__()
        self.block = block

    def forward(self, x, cos, sin, cache=None, layer=0):
        return ckpt.checkpoint(self.block, x, cos, sin, use_reentrant=False)
```

The same batch with checkpointing

```
batch 16 with activation checkpointing: activations 432 MiB, step 1050 ms
```

Checkpointing cuts the activations from 4,017 MiB to 432 MiB, by nine-tenths, and lengthens the step from 845 ms to 1,050 ms, by a quarter. That is the cost of one more forward pass, which on this machine is a quarter of a step rather than a third.

Large training runs choose what to checkpoint selectively, recomputing cheap element-wise operations and keeping the outputs of expensive matrix multiplications, to save most of the memory for a fraction of the time.

KEY IDEA

Measure before optimising. Synchronise before stopping a timer on GPU work, and account for memory as fixed state, 16 bytes per parameter with AdamW, plus activations that grow with the tokens per batch. Checkpointing removes most of the activations for a quarter to a third more time, and fusion reduces memory traffic without giving anything up.

IN PYTORCH

`torch.profiler.profile` records operations, shapes, times and memory, and exports traces for the browser's trace viewer; `torch.mps.profiler.profile` adds Instruments signposts on Apple Silicon. `torch.cuda.max_memory_allocated` reports peak memory on NVIDIA GPUs; on MPS, `torch.mps.current_allocated_memory` and `driver_allocated_memory` report what is held now. `torch.utils.checkpoint.checkpoint` implements checkpointing, and `torch.compile(mode="max-autotune")` searches for faster kernels at the cost of a longer compilation.

Reading

- Tianqi Chen, Bing Xu, Chiyuan Zhang and Carlos Guestrin, "Training Deep Nets with Sublinear Memory Cost", 2016: activation checkpointing.
- Vijay Korthikanti and others, "Reducing Activation Recomputation in Large Transformer Models", MLSys 2023: activation memory per layer, counted, and selective recomputation.
- Jason Ansel and others, "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation", ASPLOS 2024.

Labs

LAB 4.2.1 A profile of your own — 1.5 hours

Goal. Use a real profiler on a training step.

Code. `code/u4/profile.py`

1. Record three training steps with `torch.profiler` and list the ten operations with the most device or CPU time.
2. On Apple Silicon, also record them with `torch.mps.profiler` and view the timeline in Instruments; elsewhere use your vendor's tool.

DONE WHEN

- [ ] Your top-ten list accounts for most of the step's time.
- [ ] You can find one gap on the timeline where the GPU is idle, and say why.

EVIDENCE FOR THE NOTEBOOK

- The list and an annotated screenshot of the timeline.

LAB 4.2.2 Predict, then measure 1.5 hours

Goal. Plan a fine-tuning run's memory before running it.

Code. `code/u4/profile.py`

1. Predict the peak memory of fine-tuning every weight of SmolLM2-360M with AdamW at batches of about 2,048 tokens, from the memory equation and an activation estimate scaled from this chapter's per-token figure by width and depth.
2. Measure it with `uv run -m u3.lora --full --model HuggingFaceTB/SmolLM2-360M --batch-tokens 2048 --examples 1000`, which prints the largest memory in use that it sampled.

DONE WHEN

- [ ] Your prediction is within 20% of the measurement.

EVIDENCE FOR THE NOTEBOOK

- The prediction with its working, and the measurement.

LAB 4.2.3 Selective checkpointing 1.5 hours

Goal. Find a better trade than checkpointing everything.

Code. `code/u4/profile.py`

1. Checkpoint every second block, then only the feed-forward networks, and measure memory and step time for each.
2. Plot step time against activation memory for these and the two extremes.

DONE WHEN

- [ ] Your plot has four points.
- [ ] You can recommend a policy for a run that needs to halve its activation memory.

EVIDENCE FOR THE NOTEBOOK

- The plot and the recommendation.

LAB 4.2.4 The biggest batch 1 hour

Goal. Find what fits.

Code. `code/u2/pretrain.py`

1. Find the largest batch of 512-token sequences with which Chapter 2.5's model trains on your machine, with and without checkpointing.
2. Compare tokens per second at the largest batch and at the default 32.

DONE WHEN

- [ ] You have both maximum batches and the throughput at each.
- [ ] You can explain why the largest batch is or is not the fastest.

EVIDENCE FOR THE NOTEBOOK

- The batches and throughputs.

Problem set

1. Training a 7-billion-parameter model with AdamW in float32 needs how much memory before activations? With bfloat16 weights and gradients but float32 moments and master weights?

2. Activations are 500 KiB per token for an 8-layer model of width 512. Estimate them for a 32-layer model of width 4,096 with the same structure, and for a batch of one million tokens.
3. Why does checkpointing cost about a third of a training step rather than half or all of it?
4. A hook that synchronises the device after every module makes the measured total larger than the unhooked forward pass. Why?
5. Why does fusing element-wise operations save time on the M2 Pro but not FLOPs?

COMPLETION CHECKLIST

- [ ] I can time GPU work correctly and break a step down by phase and by module.
- [ ] I can write the memory equation for training and account for activations.
- [ ] I can apply activation checkpointing and measure its trade.
- [ ] I can explain fusion and what `torch.compile` gains.
- [ ] Lab 4.2.1–4.2.4 done-when criteria all hold, with evidence filed.

CHAPTER 4.3

Numerics

TIME	5 hours
YOU BUILD	A table of the floating-point formats of machine learning, from float32 to fp8; sums that stall, exponentials that overflow and gradients that underflow, each demonstrated and fixed; the error of a matrix product in each format, measured; and the speed of each on your GPU.
NEEDS	Chapters 1.3, 4.1 and 4.2.

WHY IT MATTERS

Every number in a model is approximate, and the format it is stored in sets how approximate. Fewer bits per number mean less memory, less traffic and, on hardware built for them, faster arithmetic, which is why large models train in 16 bits and increasingly in 8. They also mean less range and less precision, and the failures are silent: a sum that stops growing, a norm that becomes infinity, a gradient that becomes zero. This chapter shows each failure on purpose, so that you can recognise it, and the standard remedies, so that you can use low precision safely.

SKIP TEST

1. How many bits of exponent and mantissa do float16 and bfloat16 have, and what does each trade for the other?
2. Why does adding 0.0001 to a float16 running total eventually stop changing it?
3. What is loss scaling, and why does float16 training need it while bfloat16 training does not?
4. Why are fp8 numbers always used with a scale factor?

If all four are easy, read the sections on accumulation and on fp8, and do Labs 4.3.3 and 4.3.4.

Core ideas

Floating-point formats

A floating-point number stores a sign, an *exponent* that sets its magnitude as a power of two, and a *mantissa* of significant bits. The exponent's width sets the *range*, the largest and smallest representable magnitudes; the mantissa's sets the *precision*, the spacing between neighbouring numbers relative to their size, measured by *machine epsilon*, the gap between 1 and the next larger number.

The formats, from PyTorch's description of each — code/u4/numerics.py

```
print(
    f"{'format':<10}{'bits':>5}{'mantissa':>10}{'largest':>12}"
    f"{'smallest normal':>17}{'epsilon':>11}"
)
for name, dtype in FORMATS + (
    ("fp8 e4m3", torch.float8_e4m3fn),
    ("fp8 e5m2", torch.float8_e5m2),
):
    info = torch.finfo(dtype)
    mantissa = round(-math.log2(info.eps))
    print(
        f"{name:<10}{info.bits:5d}{mantissa:10d}{info.max:12.3g}"
        f"{info.tiny:17.3g}{info.eps:11.3g}"
    )
```

Five formats of machine learning

```
$ uv run -m u4.numerics
format     bits  mantissa     largest  smallest normal    epsilon
float32      32        23     3.4e+38         1.18e-38   1.19e-07
float16      16        10    6.55e+04          6.1e-05   0.000977
bfloat16     16         7    3.39e+38         1.18e-38    0.00781
fp8 e4m3      8         3         448           0.0156      0.125
fp8 e5m2      8         2    5.73e+04          6.1e-05       0.25
```

float32, with 23 mantissa bits, gives about seven significant decimal digits and a huge range. float16 keeps 10 mantissa bits, about three digits, but only five exponent bits: nothing above 65,504 and nothing normal below 6×10^{-5}. bfloat16 ("brain float") makes the opposite trade: float32's eight exponent bits, so float32's range, and only seven mantissa bits, about two digits. The two fp8 formats go further still: e4m3 keeps three mantissa bits and a range up to 448, e5m2 two bits and a wider range. Halving the bits halves the memory and the memory traffic of every tensor stored that way, which for the memory-bound operations of Chapter 4.1 halves their time.

Accumulation

Addition loses whatever part of the smaller number falls below the larger number's last mantissa bit. A running sum grows, its spacing grows with it, and eventually each new term is smaller than half the spacing and is lost:

A running total in each format, and the same values summed into float32 code/u4/numerics.py

```python
print("adding 0.0001 to a running total 100,000 times (exact answer 10):")
for name, dtype in FORMATS:
    total, step = torch.zeros((), dtype=dtype), torch.tensor(1e-4, dtype=dtype)
    for _ in range(100_000):
        total = total + step
    wide = torch.full((100_000,), 1e-4, dtype=dtype).sum(dtype=torch.float32)
    print(
        f"  {name:<9} total kept in {name}: {float(total):8.4f}; "
        f"accumulated in float32: {float(wide):8.4f}"
    )
```

Low-precision totals stall; a float32 accumulator does not

```
adding 0.0001 to a running total 100,000 times (exact answer 10):
  float32   total kept in float32:  10.0063; accumulated in float32:  10.0000
  float16   total kept in float16:   0.2500; accumulated in float32:  10.0017
  bfloat16  total kept in bfloat16:   0.0312; accumulated in float32:  10.0136
```

In float16 the total stops at 0.25, where the spacing between neighbouring numbers is $2^{-12} \approx 0.00024$ and 0.0001 is less than half of it; in bfloat16, with three fewer mantissa bits, it stops at 0.03. Even float32 drifts by six thousandths over 100,000 additions. Summed into a float32 accumulator, as PyTorch does internally for `sum`, the 16-bit values give nearly the right answer. The general rule is to *store* values in low precision and *accumulate* them in high precision, and matrix multiplication hardware follows it, multiplying 16-bit inputs and accumulating in 32 bits.

Range: overflow and underflow

float16's small range causes two failures. Large values overflow to infinity:

An exponential and a mean of squares, too large for float16 code/u4/numerics.py

```
half = torch.finfo(torch.float16).max
print(
    f"float16 overflows above {half:,.0f}: "
    f"exp(11) = {float(torch.tensor(11.0).half().exp()):,.0f}, "
    f"exp(12) = {float(torch.tensor(12.0).half().exp())}"
)
x = torch.randn(4096) * 300
print(
    f"mean square of 4,096 activations of size about 300: "
    f"{float((x**2).mean()):,.0f} in float32, "
    f"{float((x.half() ** 2).mean())} in float16"
)
```

Infinity from values that are not large

```
float16 overflows above 65,504: exp(11) = 59,872, exp(12) = inf
mean square of 4,096 activations of size about 300: 88,799 in float32, inf in float16
```

Activations of a few hundred are common in a large model, but their squares exceed 65,504, so a norm computed in float16 divides by infinity. This is why Chapter 2.4's RMSNorm, and softmax and loss computations generally, run in float32 whatever the model's precision. Small values underflow to zero. The gradient of a loss averaged over a large batch, flowing back through many layers, is often below float16's smallest normal number.

Small gradients in float16, with and without a scale code/u4/numerics.py

```
grads = torch.tensor([3e-4, 2e-6, 4e-8, 1e-9])
print(
    "gradients 3e-4, 2e-6, 4e-8, 1e-9 stored in float16:   ",
    ", ".join(f"{float(g):.1e}" for g in grads.half()),
)
print(
    "the same gradients scaled by 1024, stored, unscaled:",
    ", ".join(f"{float(g) / 1024:.1e}" for g in (grads * 1024).half()),
)
```

Scaling by 1,024 rescues the gradients that float16 would lose

```
gradients 3e-4, 2e-6, 4e-8, 1e-9 stored in float16:    3.0e-04, 2.0e-06, 6.0e-08, 0.0e+00
the same gradients scaled by 1024, stored, unscaled: 3.0e-04, 2.0e-06, 4.0e-08, 9.9e-10
```

Loss scaling multiplies the loss by a large factor before the backward pass, so that every gradient is multiplied by it too and stays within float16's range, and divides the gradients by the same factor, in float32, before the optimiser step. PyTorch's `GradScaler` chooses the factor dynamically: it doubles it after every 2,000 steps without an overflow, and halves it and skips the step whenever a gradient overflows to infinity. bfloat16, with float32's range, needs none of this, which is why it has become the default for training.

Precision of a matrix product

Relative error of a 512 × 512 product in each format, against float64

```
relative error of a 512 x 512 matrix product against float64:
  float32   4.1e-07
  float16   3.6e-04
  bfloat16  2.9e-03
  fp8 e4m3  3.7e-02 (inputs scaled into range, products accumulated in float32)
```

Rounding the inputs to fewer bits makes each product slightly wrong, and the errors of a sum of 512 products partly cancel. float16's product is wrong by a few parts in ten thousand, bfloat16's by a few parts in a thousand, eight times more, matching its three fewer mantissa bits. For training either is enough, because the noise of stochastic gradients is far larger. For fp8, with three mantissa bits, the error reaches a few per cent, and fp8 is usable only with care.

fp8 and scaling

fp8 e4m3's largest number is 448 and its smallest normal number 1.6×10^{-2}, a range of about 2^{15}. Most tensors' values span more than that, or sit outside it, so fp8 is always used with a *scale*: each tensor, or each block of one, is divided by a factor that maps its largest value to the format's largest, stored with the factor, and multiplied back after the arithmetic.

An fp8 matrix product with per-tensor scales, accumulated in float32 — code/u4/numerics.py

```
scale_a, scale_b = a.abs().max() / 448, b.abs().max() / 448
a8 = (a / scale_a).to(torch.float8_e4m3fn)
b8 = (b / scale_b).to(torch.float8_e4m3fn)
product = (a8.float() @ b8.float()) * scale_a * scale_b
```

Hardware with fp8 arithmetic, NVIDIA's from the Hopper generation on, doubles its throughput again over 16-bit formats. The largest recent training runs use fp8 for most of their matrix multiplications, with per-block scales and a careful choice of which operations stay in higher precision. Chapter 4.7 applies the same idea of a scaled low-precision format to model weights, where it is called quantisation.

Speed of each format

A 4096 × 4096 product in each format on the M2 Pro's GPU

```
matrix product of 4096 x 4096 on the mps device:
  float32    5.55 TFLOP/s
  float16    6.01 TFLOP/s
  bfloat16   2.91 TFLOP/s
```

On GPUs with tensor cores, 16-bit products run several times faster than float32. The M2 Pro's GPU has no separate matrix units for 16-bit formats, and its float16 products run at about float32's speed. bfloat16 runs at half that speed because on the M2 generation of Apple's GPUs each bfloat16 operation is computed in float32, with conversions on the way in and out. That is why Chapter 2.5 measured float16 autocast as only 5% faster than float32 for the whole training step, and bfloat16 as a third slower. It is also why this course trains in float32 on the M2 Pro, where 5% does not repay loss scaling and the risk of overflow. On a recent NVIDIA GPU the same code would run several times faster in bfloat16, so formats must be compared on the machine at hand.

> **KEY IDEA**
> Choose formats by range and precision: bfloat16 for training, since it has float32's range and its imprecision averages out; float16 only with loss scaling; float32 for norms, softmax, losses and every accumulator; fp8 only with scales. Then measure, because whether fewer bits are faster depends on the hardware.

> **IN PYTORCH**
> `torch.autocast(device_type, dtype=torch.bfloat16)` runs matrix multiplications and convolutions in the lower precision and keeps norms, softmax and losses in float32, without changing the model's float32 master weights. `torch.amp.GradScaler` implements dynamic loss scaling for float16. `torch.finfo(dtype)` describes any format, and `torch.float8_e4m3fn` and `torch.float8_e5m2` exist as storage types, with fused fp8 matrix multiplication on supported GPUs through `torch._scaled_mm`.

Reading

- David Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic", *ACM Computing Surveys* 23(1), 1991.
- Paulius Micikevicius and others, "Mixed Precision Training", ICLR 2018: loss scaling and float32 master weights.
- Dhiraj Kalamkar and others, "A Study of BFLOAT16 for Deep Learning Training", 2019.

- Paulius Micikevicius and others, “FP8 Formats for Deep Learning”, 2022.

Labs

LAB 4.3.1 Where a sum stalls — 1 hour

Goal. Predict and measure the stall of a running sum.

Code. `code/u4/numerics.py`

1. From each format’s mantissa width, predict the total at which adding 0.0001 stops changing it, and the totals for 0.001 and 0.01.
2. Measure all nine and compare. Then implement Kahan’s compensated summation in float16 and measure its result.

DONE WHEN

- [ ] Your nine predictions agree with the measurements to within a factor of two.
- [ ] Your Kahan sum in float16 reaches 10 to within 1%.

EVIDENCE FOR THE NOTEBOOK

- The predictions, the measurements and the code.

LAB 4.3.2 Training in float16 — 1.5 hours

Goal. See loss scaling at work.

Code. `code/u2/pretrain.py`

1. Train the `tiny` preset for 20 million tokens three ways: float32; float16 autocast with `GradScaler`; and float16 autocast with the scaler disabled.
2. Log the scaler’s scale factor and count the skipped steps.

DONE WHEN

- [ ] You have three loss curves and the scale factor’s history.
- [ ] You can explain the difference between the scaled and unscaled float16 runs from the gradients’ magnitudes.

EVIDENCE FOR THE NOTEBOOK

- The curves and the explanation.

LAB 4.3.3 Norms in low precision — 1.5 hours

Goal. Find where float16 breaks a transformer.

Code. `code/aif/gpt.py`

1. Change `RMSNorm` to compute in the input’s precision instead of float32, and run Chapter 2.5′s model in float16 on validation text.
2. Find the first layer whose activations produce infinities, and the activation size that caused them.

DONE WHEN

- [ ] You can name the layer and the value that overflowed.
- [ ] Restoring the float32 computation removes the infinities.

EVIDENCE FOR THE NOTEBOOK

- The trace and the fix.

LAB 4.3.4 fp8 with finer scales — 1 hour

Goal. Measure how scale granularity affects fp8's error.

Code. `code/u4/numerics.py`

1. Repeat the fp8 product with one scale per tensor, per row, and per block of 32 values along the inner dimension.
2. Repeat with inputs drawn from a heavy-tailed distribution, such as a Student t with three degrees of freedom.

DONE WHEN

- [ ] Your table has six relative errors.
- [ ] You can explain why finer scales matter more for heavy-tailed values.

EVIDENCE FOR THE NOTEBOOK

- The table and the explanation.

Problem set

1. How many decimal digits does a mantissa of 10 bits give? Of 7? Of 3?
2. A float16 running sum adds terms of size 1. At what total does it stop growing? A bfloat16 one?
3. Loss scaling multiplies the loss by 2^{16}. A gradient's true value is 10^{-9}. Is it representable after scaling in float16? What happens to a gradient of 10?
4. Why does mixed-precision training keep a float32 copy of the weights, when it computes in 16 bits? What would go wrong updating 16-bit weights directly with a learning rate of 10^{-5}?
5. An fp8 e4m3 tensor's values range from 10^{-3} to 10^{3}. Can one scale represent both ends? What would you do?

COMPLETION CHECKLIST

- [ ] I can describe float32, float16, bfloat16 and fp8 by range and precision.
- [ ] I can explain why accumulators stay in float32, and why norms and softmax do too.
- [ ] I can explain loss scaling and when it is needed.
- [ ] I can measure whether a format is faster on my hardware, and explain the result.
- [ ] Lab 4.3.1–4.3.4 done-when criteria all hold, with evidence filed.

CHAPTER 4.4

Kernels

TIME	9 hours
YOU BUILD	Matrix multiplication written three times as a GPU kernel (naive, tiled in shared memory, and register-blocked) and, on the Metal track, measured against the vendor's; a row-wise softmax built on subgroup reductions; and FlashAttention's forward pass, tiled over keys with an online softmax, checked against the reference and, on the Metal track, timed against attention that stores its scores and against PyTorch's own. Each kernel exists twice, in Metal, run from PyTorch, and in Vulkan's GLSL, run through the vkf library. Choose one track, or follow both.
NEEDS	Chapters 4.1 and 4.2. For the Metal track, an Apple Silicon Mac. For the Vulkan track, a GPU with a Vulkan 1.3 driver, the Vulkan SDK, CMake and Ninja; the Vulkan course's Units 1 and 2 help but are not required.

WHY IT MATTERS

Every operation in this course has run on kernels someone else wrote. Writing them yourself shows why those kernels are shaped as they are: why a matrix multiplication tiles its inputs, why a softmax works row by row, and why FlashAttention, the most important kernel of the last few years, computes attention without ever storing it. The ideas are the same on every GPU, so the chapter offers two tracks, Apple's Metal, run straight from PyTorch, and Vulkan, which runs on GPUs from every vendor and on Macs through MoltenVK. The listings set them side by side.

SKIP TEST

1. In a naive matrix multiplication kernel, how many times is each input element read from memory? How does tiling in shared memory change that?
2. What does register blocking add to tiling, and why does it help?
3. How can a softmax over a row be computed in one pass, keeping a running maximum and a running sum?
4. Why does FlashAttention read and write less memory than attention that materialises its scores?

If all four are easy, read the FlashAttention section and do Labs 4.4.3 and 4.4.4.

Core ideas

The programming model, in two vocabularies

A GPU kernel is a function run by many threads at once, each with its own index. Threads are grouped into *threadgroups*, in Metal's vocabulary, or *workgroups*, in Vulkan's, which run together on one core, can share a small fast *threadgroup* or *shared* memory, and can wait for each other at a *barrier*. Within a group, threads execute in lockstep in *SIMD groups* or *subgroups* of 32, which can exchange values directly with reductions such as a sum or a maximum.

Idea	Metal	Vulkan GLSL
Thread's index in the grid	`thread_position_in_grid`	`gl_GlobalInvocationID`
Group of threads	threadgroup	workgroup, `local_size`
Index within the group	`thread_position_in_threadgroup`	`gl_LocalInvocationID`
Group's memory	`threadgroup float t[...]`	`shared float t[...]`
Barrier	`threadgroup_barrier(...)`	`barrier()`
Lockstep subset, 32 wide here	SIMD group, `simd_sum`	subgroup, `subgroupAdd`

The Metal track compiles its kernels at run time with `torch.mps.compile_shader` and calls them on PyTorch tensors, so checking against PyTorch's own result is one line:

Metal: compiling the kernels from Python — code/u4/kernels_metal.py

```
lib = torch.mps.compile_shader((Path(__file__).parent / "kernels.metal").read_text())
```

The Vulkan track compiles GLSL to SPIR-V with `glslc` at build time, and a C++ program uploads data, dispatches the kernels through vkf, times them with timestamp queries, and checks sampled results against a CPU reference in double precision:

Vulkan: a pipeline per kernel, with sizes passed as push constants — code/kernels/vulkan/main.cpp

```
struct Kernel {
    vkf::Unique<VkPipelineLayout> layout;
    vkf::Unique<VkPipeline> pipeline;

    Kernel(const vkf::Context& ctx, VkDescriptorSetLayout set, uint32_t pushBytes,
           const char* shader) {
        const VkPushConstantRange range{VK_SHADER_STAGE_COMPUTE_BIT, 0, pushBytes};
        layout = vkf::createPipelineLayout(ctx, {set}, {range});
        auto module = vkf::loadShader(ctx, vkf::shaderPath(shader));
        pipeline = vkf::createComputePipeline(
            ctx, {.layout = layout.get(), .module = module.get(), .name = shader});
    }

    template <class Push>
    void record(VkCommandBuffer cmd, VkDescriptorSet set, const Push& push, uint32_t x,
                uint32_t y) const {
        vkCmdBindPipeline(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, pipeline.get());
        vkCmdBindDescriptorSets(cmd, VK_PIPELINE_BIND_POINT_COMPUTE, layout.get(), 0, 1, &set,
                                0, nullptr);
        vkCmdPushConstants(cmd, layout.get(), VK_SHADER_STAGE_COMPUTE_BIT, 0, sizeof(Push),
                           &push);
        vkCmdDispatch(cmd, x, y, 1);
    }
};
```

Naive matrix multiplication

The naive kernel runs one thread per output element, each computing the dot product of a row and a column:

Metal: one output element per thread — code/u4/kernels.metal

```
kernel void matmul_naive(device const float* a [[buffer(0)]],
                         device const float* b [[buffer(1)]],
                         device float* c [[buffer(2)]], constant uint& n [[buffer(3)]],
                         constant uint& k [[buffer(4)]], constant uint& m [[buffer(5)]],
                         uint2 id [[thread_position_in_grid]]) {
    uint row = id.y, col = id.x;
    if (row >= n || col >= m) return;
    float sum = 0.0f;
    for (uint p = 0; p < k; ++p) sum += a[row * k + p] * b[p * m + col];
    c[row * m + col] = sum;
}
```

Vulkan: the same kernel in GLSL — code/kernels/vulkan/matmul_naive.comp

```
layout(local_size_x = 16, local_size_y = 16) in;
layout(std430, binding = 0) readonly buffer A { float a[]; };
layout(std430, binding = 1) readonly buffer B { float b[]; };
layout(std430, binding = 2) writeonly buffer C { float c[]; };
layout(push_constant) uniform Sizes { uint n, k, m; };

void main() {
    uint row = gl_GlobalInvocationID.y, col = gl_GlobalInvocationID.x;
    if (row ⩾ n || col ⩾ m) return;
    float sum = 0.0;
    for (uint p = 0; p < k; ++p) sum += a[row * k + p] * b[p * m + col];
    c[row * m + col] = sum;
}
```

Each thread reads $2k$ floats to perform $2k$ FLOPs, an intensity of $1/4$, and threads that need the same row or column read it again. Neighbouring threads read neighbouring columns of B, which the memory system combines into wide transactions, so the kernel runs faster than its intensity suggests, although still far from the roofline's ridge.

Tiling in shared memory

A threadgroup of 16×16 threads computes a 16×16 tile of the output. It walks along the inner dimension 16 at a time: each thread loads one element of A's tile and one of B's into shared memory, the group waits at a barrier, and each thread then performs 16 multiply-adds from shared memory before the next tiles are loaded. Every element loaded from main memory is now used 16 times.

Metal: tiles of A and B in threadgroup memory — code/u4/kernels.metal

```
constant uint TILE = 16;

kernel void matmul_tiled(device const float* a [[buffer(0)]],
                         device const float* b [[buffer(1)]],
                         device float* c [[buffer(2)]], constant uint& n [[buffer(3)]],
                         constant uint& k [[buffer(4)]], constant uint& m [[buffer(5)]],
                         uint2 id [[thread_position_in_grid]],
                         uint2 local [[thread_position_in_threadgroup]]) {
    threadgroup float ta[TILE][TILE];
    threadgroup float tb[TILE][TILE];
    uint row = id.y, col = id.x;
    float sum = 0.0f;
    for (uint start = 0; start < k; start += TILE) {
        bool inA = row < n && start + local.x < k, inB = col < m && start + local.y < k;
        ta[local.y][local.x] = inA ? a[row * k + start + local.x] : 0.0f;
        tb[local.y][local.x] = inB ? b[(start + local.y) * m + col] : 0.0f;
        threadgroup_barrier(mem_flags::mem_threadgroup);
        for (uint p = 0; p < TILE; ++p) sum += ta[local.y][p] * tb[p][local.x];
        threadgroup_barrier(mem_flags::mem_threadgroup);
    }
    if (row < n && col < m) c[row * m + col] = sum;
}
```

Vulkan: the same, in shared memory — code/kernels/vulkan/matmul_tiled.comp

```
const uint TILE = 16;
layout(local_size_x = 16, local_size_y = 16) in;
layout(std430, binding = 0) readonly buffer A { float a[]; };
layout(std430, binding = 1) readonly buffer B { float b[]; };
layout(std430, binding = 2) writeonly buffer C { float c[]; };
layout(push_constant) uniform Sizes { uint n, k, m; };
shared float ta[TILE][TILE];
shared float tb[TILE][TILE];

void main() {
    uint row = gl_GlobalInvocationID.y, col = gl_GlobalInvocationID.x;
    uvec2 local = gl_LocalInvocationID.xy;
    float sum = 0.0;
    for (uint start = 0; start < k; start += TILE) {
        bool inA = row < n && start + local.x < k, inB = col < m && start + local.y < k;
        ta[local.y][local.x] = inA ? a[row * k + start + local.x] : 0.0;
        tb[local.y][local.x] = inB ? b[(start + local.y) * m + col] : 0.0;
        barrier();
        for (uint p = 0; p < TILE; ++p) sum += ta[local.y][p] * tb[p][local.x];
        barrier();
    }
    if (row < n && col < m) c[row * m + col] = sum;
}
```

The two barriers per step are both necessary: the first stops threads computing with a tile that others have not finished loading, the second stops them overwriting a tile that others are still reading.

Register blocking

Shared memory is fast, but each multiply-add above still reads two values from it. *Register blocking* has each thread compute a 4×4 block of outputs: per step of the inner dimension, it loads four values of A and four of B into registers and performs sixteen multiply-adds, so each value read from shared memory is used four times. A threadgroup of 16×16 threads then covers a 64×64 output tile.

Metal: 4×4 outputs per thread from a 64×64 tile — code/u4/kernels.metal

```
constant uint BLOCK = 64;
constant uint WORK = 4;

kernel void matmul_blocked(device const float* a [[buffer(0)]],
                           device const float* b [[buffer(1)]],
                           device float* c [[buffer(2)]], constant uint& n [[buffer(3)]],
                           constant uint& k [[buffer(4)]], constant uint& m [[buffer(5)]],
                           uint2 group [[threadgroup_position_in_grid]],
                           uint2 local [[thread_position_in_threadgroup]]) {
    threadgroup float ta[BLOCK][TILE];
    threadgroup float tb[TILE][BLOCK];
    float acc[WORK][WORK] = {};
    uint row0 = group.y * BLOCK + local.y * WORK, col0 = group.x * BLOCK + local.x * WORK;
    uint linear = local.y * (BLOCK / WORK) + local.x;
    for (uint start = 0; start < k; start += TILE) {
        for (uint i = linear; i < BLOCK * TILE; i += (BLOCK / WORK) * (BLOCK / WORK)) {
            uint r = i / TILE, p = i % TILE;
            uint gr = group.y * BLOCK + r;
            ta[r][p] = gr < n && start + p < k ? a[gr * k + start + p] : 0.0f;
            uint q = i / BLOCK, col = i % BLOCK;
            uint gc = group.x * BLOCK + col;
            tb[q][col] = gc < m && start + q < k ? b[(start + q) * m + gc] : 0.0f;
        }
        threadgroup_barrier(mem_flags::mem_threadgroup);
        for (uint p = 0; p < TILE; ++p) {
            float av[WORK], bv[WORK];
            for (uint i = 0; i < WORK; ++i) av[i] = ta[local.y * WORK + i][p];
            for (uint j = 0; j < WORK; ++j) bv[j] = tb[p][local.x * WORK + j];
            for (uint i = 0; i < WORK; ++i)
                for (uint j = 0; j < WORK; ++j) acc[i][j] = fma(av[i], bv[j], acc[i][j]);
        }
        threadgroup_barrier(mem_flags::mem_threadgroup);
```

```
    }
    for (uint i = 0; i < WORK; ++i)
        for (uint j = 0; j < WORK; ++j)
            if (row0 + i < n && col0 + j < m) c[(row0 + i) * m + col0 + j] = acc[i][j];
}
```

Vulkan: the same blocking — code/kernels/vulkan/matmul_blocked.comp

```
const uint TILE = 16, BLOCK = 64, WORK = 4;
layout(local_size_x = 16, local_size_y = 16) in;
layout(std430, binding = 0) readonly buffer A { float a[]; };
layout(std430, binding = 1) readonly buffer B { float b[]; };
layout(std430, binding = 2) writeonly buffer C { float c[]; };
layout(push_constant) uniform Sizes { uint n, k, m; };
shared float ta[BLOCK][TILE];
shared float tb[TILE][BLOCK];

void main() {
    uvec2 group = gl_WorkGroupID.xy, local = gl_LocalInvocationID.xy;
    float acc[WORK][WORK];
    for (uint i = 0; i < WORK; ++i)
        for (uint j = 0; j < WORK; ++j) acc[i][j] = 0.0;
    uint linear = gl_LocalInvocationIndex;
    for (uint start = 0; start < k; start += TILE) {
        for (uint i = linear; i < BLOCK * TILE; i += 256) {
            uint r = i / TILE, p = i % TILE, gr = group.y * BLOCK + r;
            ta[r][p] = gr < n && start + p < k ? a[gr * k + start + p] : 0.0;
            uint q = i / BLOCK, col = i % BLOCK, gc = group.x * BLOCK + col;
            tb[q][col] = gc < m && start + q < k ? b[(start + q) * m + gc] : 0.0;
        }
        barrier();
        for (uint p = 0; p < TILE; ++p) {
            float av[WORK], bv[WORK];
            for (uint i = 0; i < WORK; ++i) av[i] = ta[local.y * WORK + i][p];
            for (uint j = 0; j < WORK; ++j) bv[j] = tb[p][local.x * WORK + j];
            for (uint i = 0; i < WORK; ++i)
                for (uint j = 0; j < WORK; ++j) acc[i][j] = fma(av[i], bv[j], acc[i][j]);
        }
        barrier();
    }
    uint row0 = group.y * BLOCK + local.y * WORK, col0 = group.x * BLOCK + local.x * WORK;
    for (uint i = 0; i < WORK; ++i)
        for (uint j = 0; j < WORK; ++j)
            if (row0 + i < n && col0 + j < m) c[(row0 + i) * m + col0 + j] = acc[i][j];
}
```

The Metal track launches each kernel with a grid matching its tile:

Metal: launching the three kernels, and PyTorch's for comparison — code/u4/kernels_metal.py

```
c = torch.empty(n, n, device=dev)
launches = {
    "naive": lambda: lib.matmul_naive(
        a, b, c, n, n, n, threads=(n, n), group_size=(16, 16)
    ),
    "tiled": lambda: lib.matmul_tiled(
        a, b, c, n, n, n, threads=(up(n, 16), up(n, 16)), group_size=(16, 16)
    ),
    "register-blocked": lambda: lib.matmul_blocked(
        a, b, c, n, n, n, threads=(up(n, 64) // 4, up(n, 64) // 4), group_size=(16, 16)
    ),
    "PyTorch (MPS)": lambda: torch.mm(a, b, out=c),
}
```

Metal track: GFLOP/s on the M2 Pro

```
$ uv run -m u4.kernels_metal
matrix product           n = 1024     n = 2048     n = 4096   (GFLOP/s)
naive                         386          444          430
tiled                         699          720          733
register-blocked            1,448        1,762        1,787
PyTorch (MPS)               3,560        5,256        5,545
```

Vulkan track: GFLOP/s on the M2 Pro through MoltenVK

```
$ ./build/bin/aif_kernels
vkf: Apple M2 Pro (integrated GPU), Vulkan 1.3.357, MoltenVK 1.4.2
vkf: validation off, synchronisation validation off
matrix product           n = 1024     n = 2048     n = 4096   (GFLOP/s)
naive                         401          451          425
tiled                         749          730          736
register-blocked             1589         1751         1751
```

On both tracks the naive kernel reaches about 0.4 TFLOP/s, reading every operand from memory for every product. Tiling through shared memory nearly doubles it, to about 0.73, and register blocking more than doubles that again, to about 1.8 TFLOP/s at the larger sizes. There the two tracks agree within a few per cent, as expected, since MoltenVK translates the Vulkan kernels into Metal ones and the hardware underneath is the same. The vendor's kernel reaches 5.5 TFLOP/s, three times the blocked kernel's. It uses the GPU's SIMD-group matrix instructions, which multiply small matrices held across a SIMD group's registers in a single operation, together with larger tiles and loads overlapped with arithmetic. The Metal option of Lab 4.4.3 tries the matrix instructions.

Softmax, one row per threadgroup

A softmax over a row needs the row's maximum and the sum of exponentials, both reductions. One threadgroup handles one row: each thread reduces a strided share of the row, the SIMD group or subgroup combines its 32 threads' values in registers, the first thread of each writes its result to shared memory, and one more reduction across the groups gives the row's value.

Metal: a row softmax with SIMD-group reductions `code/u4/kernels.metal`

```
kernel void softmax_rows(device const float* x [[buffer(0)]], device float* y [[buffer(1)]],
                         constant uint& cols [[buffer(2)]],
                         uint row [[threadgroup_position_in_grid]],
                         uint t [[thread_position_in_threadgroup]],
                         uint width [[threads_per_threadgroup]],
                         uint lane [[thread_index_in_simdgroup]],
                         uint sg [[simdgroup_index_in_threadgroup]]) {
    threadgroup float partial[32];
    device const float* in = x + row * cols;
    device float* out = y + row * cols;
    float peak = -INFINITY;
    for (uint i = t; i < cols; i += width) peak = max(peak, in[i]);
    peak = simd_max(peak);
    if (lane == 0) partial[sg] = peak;
    threadgroup_barrier(mem_flags::mem_threadgroup);
    peak = simd_max(lane < (width + 31) / 32 ? partial[lane] : -INFINITY);
    threadgroup_barrier(mem_flags::mem_threadgroup);
    float total = 0.0f;
    for (uint i = t; i < cols; i += width) total += exp(in[i] - peak);
    total = simd_sum(total);
    if (lane == 0) partial[sg] = total;
    threadgroup_barrier(mem_flags::mem_threadgroup);
    total = simd_sum(lane < (width + 31) / 32 ? partial[lane] : 0.0f);
    for (uint i = t; i < cols; i += width) out[i] = exp(in[i] - peak) / total;
}
```

Vulkan: the same with subgroup operations `code/kernels/vulkan/softmax.comp`

```
layout(local_size_x = 256) in;
layout(std430, binding = 0) readonly buffer X { float x[]; };
layout(std430, binding = 1) writeonly buffer Y { float y[]; };
layout(push_constant) uniform Sizes { uint cols; };
shared float partial[32];

float groupReduce(float value, bool isMax) {
    value = isMax ? subgroupMax(value) : subgroupAdd(value);
    if (gl_SubgroupInvocationID == 0) partial[gl_SubgroupID] = value;
    barrier();
    float mine = gl_SubgroupInvocationID < gl_NumSubgroups ? partial[gl_SubgroupInvocationID]
                                                           : (isMax ? -1.0 / 0.0 : 0.0);
    value = isMax ? subgroupMax(mine) : subgroupAdd(mine);
    barrier();
    return value;
}

void main() {
    uint base = gl_WorkGroupID.x * cols, t = gl_LocalInvocationID.x;
    float peak = -1.0 / 0.0;
    for (uint i = t; i < cols; i += 256) peak = max(peak, x[base + i]);
    peak = groupReduce(peak, true);
    float total = 0.0;
    for (uint i = t; i < cols; i += 256) total += exp(x[base + i] - peak);
    total = groupReduce(total, false);
    for (uint i = t; i < cols; i += 256) y[base + i] = exp(x[base + i] - peak) / total;
}
```

Metal track: softmax bandwidth

```
softmax over 4,096 rows of 8,192: yours 130 GB/s, PyTorch's 112 GB/s
```

Vulkan track: softmax bandwidth

```
softmax over 4096 rows of 8192: 146 GB/s (largest error 1.2e-08)
```

The softmax is memory-bound, so it is measured by bandwidth, counting one read of the row and one write of the result. This kernel reads the row three times, for the maximum, the sum and the output. A better one keeps the row in registers or shared memory and reads it once, or, as the next section shows, avoids the separate maximum altogether. Even so, this kernel beats PyTorch's general softmax on this shape, 130 GB/s against 112, and the Vulkan version reaches 146.

FlashAttention

Attention as written in Chapter 2.3 computes the $T \times T$ scores, writes them to memory, reads them back for the softmax, writes the weights, and reads them again to multiply by the values. For long sequences these scores dominate both memory traffic and memory use. FlashAttention never stores them: it processes the keys in blocks and computes the softmax as it goes, using an *online* softmax.

> **DERIVATION**
> For one query, keep a running maximum m, a running sum ℓ of $\exp(s_j - m)$ and an accumulator o of $\sum_j \exp(s_j - m)v_j$. When a new score s arrives with value v, the new maximum is $m' = \max(m, s)$; the old sum and accumulator were computed relative to m, so rescale them by $\exp(m - m')$, then add the new term: $\ell' = \ell e^{m-m'} + e^{s-m'}$, $o' = oe^{m-m'} + e^{s-m'}v$. After the last key, o/ℓ is exactly the softmax-weighted average, computed in one pass, with no score stored.

Each threadgroup takes 64 queries, one per thread. The group loads blocks of 32 keys and values into shared memory, and each thread updates its query's running maximum, sum and accumulator from the block. Causal masking stops each query at its own position, and a threadgroup stops loading key blocks after its last query's position, which skips the masked half of the work.

Metal: FlashAttention's forward pass, one query per thread, blocks of keys in threadgroup memory — code/u4/kernels.metal

```
constant uint KEYS = 32;
constant uint HEAD = 64;

kernel void flash_attention(device const float* q [[buffer(0)]],
                            device const float* k [[buffer(1)]],
                            device const float* v [[buffer(2)]], device float* o [[buffer(3)]],
                            constant uint& length [[buffer(4)]],
                            constant float& scale [[buffer(5)]],
                            uint2 group [[threadgroup_position_in_grid]],
                            uint2 local [[thread_position_in_threadgroup]],
                            uint2 size [[threads_per_threadgroup]]) {
    threadgroup float kb[KEYS][HEAD];
    threadgroup float vb[KEYS][HEAD];
    uint t = local.x, width = size.x;
    uint head = group.y, row = group.x * width + t;
    device const float* qh = q + head * length * HEAD;
    float qr[HEAD], acc[HEAD];
    for (uint d = 0; d < HEAD; ++d) {
        qr[d] = row < length ? qh[row * HEAD + d] * scale : 0.0f;
        acc[d] = 0.0f;
    }
    float best = -INFINITY, total = 0.0f;
    uint last = min(length, (group.x + 1) * width);
    for (uint start = 0; start < last; start += KEYS) {
        for (uint i = t; i < KEYS * HEAD; i += width) {
            uint key = start + i / HEAD;
            uint at = (head * length + key) * HEAD + i % HEAD;
            kb[i / HEAD][i % HEAD] = key < length ? k[at] : 0.0f;
            vb[i / HEAD][i % HEAD] = key < length ? v[at] : 0.0f;
        }
        threadgroup_barrier(mem_flags::mem_threadgroup);
        for (uint j = 0; j < KEYS && start + j <= row && row < length; ++j) {
            float s = 0.0f;
            for (uint d = 0; d < HEAD; ++d) s += qr[d] * kb[j][d];
            float next = max(best, s);
            float shrink = exp(best - next), weight = exp(s - next);
            total = total * shrink + weight;
            for (uint d = 0; d < HEAD; ++d) acc[d] = acc[d] * shrink + weight * vb[j][d];
            best = next;
        }
        threadgroup_barrier(mem_flags::mem_threadgroup);
    }
    if (row < length)
        for (uint d = 0; d < HEAD; ++d) o[(head * length + row) * HEAD + d] = acc[d] / total;
}
```

Vulkan: the same kernel — code/kernels/vulkan/flash.comp

```
const uint KEYS = 32, HEAD = 64;
layout(local_size_x = 64) in;
layout(std430, binding = 0) readonly buffer Q { float q[]; };
layout(std430, binding = 1) readonly buffer K { float k[]; };
layout(std430, binding = 2) readonly buffer V { float v[]; };
layout(std430, binding = 3) writeonly buffer O { float o[]; };
layout(push_constant) uniform Sizes { uint len; float scale; };
shared float kb[KEYS][HEAD];
shared float vb[KEYS][HEAD];

void main() {
    uint t = gl_LocalInvocationID.x, width = gl_WorkGroupSize.x;
    uint head = gl_WorkGroupID.y, row = gl_WorkGroupID.x * width + t;
    float qr[HEAD], acc[HEAD];
    for (uint d = 0; d < HEAD; ++d) {
        qr[d] = row < len ? q[(head * len + row) * HEAD + d] * scale : 0.0;
        acc[d] = 0.0;
    }
    float best = -1.0 / 0.0, total = 0.0;
    uint last = min(len, (gl_WorkGroupID.x + 1) * width);
```

```
    for (uint start = 0; start < last; start += KEYS) {
        for (uint i = t; i < KEYS * HEAD; i += width) {
            uint key = start + i / HEAD;
            kb[i / HEAD][i % HEAD] = key < len ? k[(head * len + key) * HEAD + i % HEAD] : 0.0;
            vb[i / HEAD][i % HEAD] = key < len ? v[(head * len + key) * HEAD + i % HEAD] : 0.0;
        }
        barrier();
        for (uint j = 0; j < KEYS && start + j ⩽ row && row < len; ++j) {
            float s = 0.0;
            for (uint d = 0; d < HEAD; ++d) s += qr[d] * kb[j][d];
            float next = max(best, s);
            float shrink = exp(best - next), weight = exp(s - next);
            total = total * shrink + weight;
            for (uint d = 0; d < HEAD; ++d) acc[d] = acc[d] * shrink + weight * vb[j][d];
            best = next;
        }
        barrier();
    }
    if (row < len)
        for (uint d = 0; d < HEAD; ++d) o[(head * len + row) * HEAD + d] = acc[d] / total;
}
```

Metal: materialised attention, your kernel and PyTorch's code/u4/kernels_metal.py

```
mask = torch.ones(length, length, dtype=torch.bool, device=dev).tril()

def materialised():
    scores = (q @ k.transpose(-1, -2)) / math.sqrt(d)
    return scores.masked_fill(~mask, float("-inf")).softmax(-1) @ v

flash = lambda: lib.flash_attention(
    q,
    k,
    v,
    out,
    length,
    1 / math.sqrt(d),
    threads=(up(length, 64), heads),
    group_size=(64, 1),
)
sdpa = lambda: F.scaled_dot_product_attention(q, k, v, is_causal=True)
```

Metal track: causal attention by length

```
causal attention, 8 heads of 64: milliseconds
  length  materialised  flash, yours  PyTorch SDPA   scores held
     512           1.3           3.3           0.4         8 MiB
    1024           2.3           8.1           0.6        32 MiB
    2048           7.6          25.5           1.7       128 MiB
    4096          30.3          92.5           5.9       512 MiB
```

Vulkan track: the flash kernel by length

```
causal flash attention, 8 heads of 64:
  length  512:    3.14 ms (largest error 1.3e-07)
  length 1024:    9.74 ms (largest error 1.1e-07)
  length 2048:   33.20 ms (largest error 1.4e-07)
  length 4096:  119.39 ms (largest error 1.1e-07)
PASS every kernel matches its CPU reference
```

Materialised attention holds the whole matrix of scores, 512 MiB at a length of 4,096 for eight heads, and would hold gigabytes at the lengths of current models. The flash kernel holds none. Its time grows with the square of the length, as attention's must, but it is two and a half to three and a half times slower than materialised attention, whose two products run on Apple's tuned matrix kernels. It is up to sixteen times slower than PyTorch's fused kernel, which has both advantages, and beyond length 512 the Vulkan track's kernel is slower again. The algorithm is correct, and the gap comes from the simple implementation.

One query per thread computes each inner product with scalar arithmetic, and every group of 64 queries reloads every key block, while the vendor's kernel uses the matrix instructions, gives each thread several queries and keeps more of each block in registers. Lab 4.4.3 closes part of the gap. A kernel for training also needs the backward pass, which recomputes the scores block by block from the maxima and sums saved by the forward pass, trading arithmetic for memory again.

> **KEY IDEA**
> A fast kernel moves each byte from memory as few times as possible: tiling reuses inputs from shared memory, register blocking reuses them from registers, and FlashAttention's online softmax avoids writing the attention scores at all. In each case the arithmetic is the same and the memory traffic is lower.

> **IN PYTORCH**
> Production kernels are written in CUDA, in Triton, a Python-embedded language that compiles tile-level programs for NVIDIA and AMD GPUs, or in Metal and its Metal Performance Shaders on Apple hardware. FlashAttention's own implementations use the GPUs' matrix units, process several queries per thread, and include a backward pass that recomputes the scores rather than storing them. PyTorch's `scaled_dot_product_attention` dispatches to such kernels where they exist.

Reading

- Tri Dao, Daniel Fu, Stefano Ermon, Atri Rudra and Christopher Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness", NeurIPS 2022.
- Maxim Milakov and Natalia Gimelshein, "Online Normalizer Calculation for Softmax", 2018.
- Simon Boehm, "How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance: a Worklog", 2022.
- Apple, *Metal Shading Language Specification*, sections on threadgroup memory and SIMD-group functions; and the Vulkan course, Chapters 2.1 to 2.5, for the Vulkan track.

Labs

LAB 4.4.1 Tile sizes — 2 hours

Goal. Map the tiled kernel's speed against its tile size.

Code. `code/u4/kernels.metal or code/kernels/vulkan`

1. In your track, make the tiled kernel's tile size a parameter and measure tiles of 8, 16 and 32 at $n = 2048$.
2. For the register-blocked kernel, try 2×2, 4×4 and 8×4 outputs per thread.

DONE WHEN

- [ ] Your table has six measurements, all checked against the reference.
- [ ] You can explain the best of each in terms of reuse and of the registers and shared memory each thread needs.

EVIDENCE FOR THE NOTEBOOK

- The table and the explanation.

LAB 4.4.2 A one-pass softmax — 1.5 hours

Goal. Halve the softmax's memory traffic.

Code. `code/u4/kernels.metal or code/kernels/vulkan/softmax.comp`

1. Use the online softmax to compute each thread's maximum and sum in one pass over its share, combining partial results with the rescaling rule.
2. Then keep the row in registers between the reduction and the output, so the row is read once.

DONE WHEN

- [ ] Both versions match the reference.
- [ ] Your measured bandwidth rises, and you can relate the gain to the bytes saved.

EVIDENCE FOR THE NOTEBOOK

- The kernels and the measurements.

LAB 4.4.3 A faster flash kernel — 3 hours

Goal. Close part of the gap to the vendor's attention.

Code. `code/u4/kernels.metal or code/kernels/vulkan/flash.comp`

1. Change the flash kernel so that each thread handles two queries, sharing each loaded key and value between them, and try key blocks of 16 and 64.
2. On the Metal track, optionally replace the inner products with `simdgroup_matrix` operations.

DONE WHEN

- [ ] Your kernel still passes the program's check against the reference.
- [ ] It is faster than the original at length 4,096, by a measured factor.

EVIDENCE FOR THE NOTEBOOK

- The kernel, the check and the timings.

LAB 4.4.4 The other track — 1.5 hours

Goal. Port one kernel between Metal and Vulkan.

Code. `code/u4/kernels.metal and code/kernels/vulkan`

1. Port your best kernel from Lab 4.4.1 or 4.4.3 to the other track.
2. Compare the two versions' speeds on the same machine and explain any difference.

DONE WHEN

- [ ] The ported kernel passes its track's checks.
- [ ] You can account for the difference in speed, or its absence.

EVIDENCE FOR THE NOTEBOOK

- Both kernels and the comparison.

Problem set

1. For the tiled kernel with tiles of $t \times t$, how many bytes are read from main memory per output element, and what is the kernel's intensity as a function of t?
2. Why does the register-blocked kernel's threadgroup cover a 64×64 tile with only 256 threads, and what limits making the blocks larger?
3. Show that the online softmax's rescaling gives exactly the same result as the two-pass softmax, in exact arithmetic.

4. Materialised attention for 8 heads at 4,096 tokens holds how many MiB of scores in float32? How much does FlashAttention hold?
5. Why does causal FlashAttention do about half the work of non-causal, while the materialised version as written does the full work and masks half of it?

COMPLETION CHECKLIST

- ☐ I can write, launch and check a GPU kernel in Metal or Vulkan.
- ☐ I can explain tiling and register blocking in terms of reuse and memory traffic.
- ☐ I can implement reductions with SIMD-group or subgroup operations.
- ☐ I can derive the online softmax and implement FlashAttention's forward pass.
- ☐ Lab 4.4.1–4.4.4 done-when criteria all hold, with evidence filed.

CHAPTER 4.5

Parallel training

TIME	7 hours
YOU BUILD	A ring all-reduce written with point-to-point messages and checked against the library's; data-parallel training across four processes that matches one-process training; ZeRO's sharded optimiser state; a tensor-parallel feed-forward layer split by columns and rows; a two-stage pipeline with micro-batches; and the communication cost of each, worked out for a model of seven billion parameters.
NEEDS	Chapters 2.4, 4.1 and 4.2.

WHY IT MATTERS

One accelerator cannot train a large model in a useful time: a 7-billion-parameter model's weights, gradients and optimiser state alone need over 100 GB, more than most GPUs hold, and its training run needs more arithmetic than one GPU could do in a year. Every large model is therefore trained across many devices, by combining a few strategies, each a different way of dividing the work and each paid for in communication. This chapter implements all of them across processes on one machine, where the logic is identical to a cluster's and only the speeds differ, so that you can reason about the choices large training runs make.

SKIP TEST

1. What does an all-reduce compute, and how many bytes does each of N processes send in a ring all-reduce of S bytes?
2. How much memory does training a 7-billion-parameter model with Adam need before activations, and what do ZeRO's three stages shard?
3. How does tensor parallelism split a feed-forward network so that its forward pass needs only one all-reduce?
4. What fraction of the time are the stages of a pipeline idle with p stages and m micro-batches?

If all four are easy, read the sections on ZeRO and on costs at scale, and do Labs 4.5.3 and 4.5.4.

Core ideas

Why and how to divide the work

Chapter 4.2's memory equation gives the minimum. With AdamW in float32, every parameter costs 16 bytes for its weight, gradient and two moments, so 7 billion parameters need 112 GB before a single activation, more than an 80 GB H100 holds. Time is the other limit. Chapter 2.4's rule of 6 FLOPs per parameter per token puts a 7-billion-parameter model trained on two trillion tokens at 8.4×10^{22} FLOPs, nearly nine years of one GPU running at 300 TFLOP/s.

There are three basic ways to divide training among devices. *Data parallelism* gives every device the whole model and a different slice of each batch. *Tensor parallelism* splits each layer's matrices among devices, which then cooperate on every token. *Pipeline parallelism* gives each device a contiguous range of layers, and passes activations along. A fourth technique, *sharding*, makes data parallelism store each piece of state on only one device. Large runs combine them, and each needs a different pattern of communication.

Collectives

Devices communicate through *collective* operations in which every process of a group takes part. A *broadcast* copies one process's tensor to all. A *reduce* sums all processes' tensors onto one, and an *all-reduce* leaves the sum on every process. A *reduce-scatter* leaves each process with the sum of one slice, and an *all-gather* leaves every process with every process's slice. PyTorch's `torch.distributed` provides them over several backends: NCCL between NVIDIA GPUs, and gloo, used here, between CPU processes. This chapter's programs start four processes on one machine, each with its own copy of PyTorch, connected through the loopback network.

The ring all-reduce

The simplest all-reduce sends every tensor to one process, which sums them and sends the result back, so that process sends and receives N times the data while the others idle. A *ring* does better. Arrange the processes in a ring and split each tensor into N chunks. In $N-1$ steps of a *reduce-scatter*, each process sends one chunk to its right-hand neighbour and adds the chunk arriving from its left into its own; afterwards each process holds the complete sum of one chunk. In $N-1$ steps of an *all-gather*, the completed chunks travel round the ring until every process has all of them.

A ring all-reduce from sends and receives — code/u4/parallel.py

```
def ring_all_reduce(t: torch.Tensor) -> None:
    world, rank = dist.get_world_size(), dist.get_rank()
    chunks = list(t.view(-1).tensor_split(world))
    right, left = (rank + 1) % world, (rank - 1) % world
    for step in range(world - 1):
        send, receive = (rank - step) % world, (rank - step - 1) % world
        incoming = torch.empty_like(chunks[receive])
        request = dist.isend(chunks[send].clone(), right)
        dist.recv(incoming, left)
        request.wait()
        chunks[receive] += incoming
    for step in range(world - 1):
        send, receive = (rank - step + 1) % world, (rank - step) % world
        incoming = torch.empty_like(chunks[receive])
        request = dist.isend(chunks[send].clone(), right)
        dist.recv(incoming, left)
        request.wait()
        chunks[receive].copy_(incoming)
```

Each process sends $2(N-1)/N$ times the tensor's size, which tends to twice the size however many processes there are, and all links are busy all the time. No algorithm can send less, so the ring is optimal in bandwidth. Its weakness is latency: it takes $2(N-1)$ sequential steps, which tree-shaped algorithms reduce for small tensors.

The ring against gloo's all-reduce, four processes

```
$ uv run -m u4.parallel
all-reduce of 4 MiB, 4 processes: ring   34.3 ms, gloo   17.5 ms, difference 9.5e-07
all-reduce of 64 MiB, 4 processes: ring  341.1 ms, gloo  256.5 ms, difference 1.9e-06
```

The ring's sums agree with the library's to within rounding, since the two add the same numbers in different orders, and it runs within a small factor of the library's speed. Gloo's implementation is a carefully tuned version of the same algorithm, with chunks pipelined so that sending one overlaps adding another.

Data parallelism

In data-parallel training, each process holds the whole model, computes gradients on its own slice of the batch, and all-reduces them, so every process steps its optimiser with the same averaged gradient. Because the gradient of a mean over a batch is the mean of the gradients over its slices, this is exactly one-process training on the whole batch, as long as the processes start from the same weights.

Data parallelism: local gradients, one all-reduce, identical steps `code/u4/parallel.py`

```
def experiment_data_parallel(
    rank: int, world: int, out: dict, steps: int = 20, rows: int = 32
) → None:
    torch.manual_seed(0)
    model = gpt.GPT(TINY)
    opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
    share = rows // world
    start = time.perf_counter()
    for step in range(steps):
        x = batch(step, rows)[rank * share : (rank + 1) * share]
        _, loss = model(x[:, :-1], x[:, 1:])
        opt.zero_grad()
        loss.backward()
        flat = torch.cat([p.grad.view(-1) for p in model.parameters()])
        ring_all_reduce(flat)
        flat /= world
        offset = 0
        for p in model.parameters():
            p.grad.copy_(flat[offset : offset + p.numel()].view_as(p))
            offset += p.numel()
        opt.step()
```

Four processes match one

```
data parallel, 4 processes, 20 steps: parameters within 3.3e-05 of one process
```

The parameters after 20 steps differ from one-process training by 3×10^{-5}. The difference is rounding: the processes add the same numbers in a different order, and Adam's division by small second moments enlarges the differences a little. Production systems overlap the all-reduce with the backward pass, sending each layer's gradients as soon as they exist, in *buckets* of a few megabytes, so that communication hides behind computation.

ZeRO: sharding the state

Data parallelism keeps a full copy of the optimiser's state on every process, which wastes memory, since each process needs to update only the parameters it is responsible for. *ZeRO*, the Zero Redundancy Optimizer, removes the redundancy in stages. Stage 1 shards the optimiser's state: after the gradients are summed, each process updates only its slice of the parameters, then all-gathers the updated slices. Stage 2 also shards the gradients, using a reduce-scatter instead of an all-reduce, and stage 3 shards the parameters themselves, gathering each layer's weights just before they are needed and discarding them afterwards.

ZeRO stage 1: Adam's state for one slice per process `code/u4/parallel.py`

```
def experiment_zero(rank: int, world: int, out: dict, steps: int = 20, rows: int = 32) → None:
    torch.manual_seed(0)
    model = gpt.GPT(TINY)
    params = list(model.parameters())
    flat = torch.cat([p.detach().view(-1) for p in params])
    shard = flat.tensor_split(world)[rank].clone().requires_grad_()
    opt = torch.optim.AdamW([shard], lr=1e-3)
    sizes = [len(s) for s in flat.tensor_split(world)]
    share = rows // world
    for step in range(steps):
        x = batch(step, rows)[rank * share : (rank + 1) * share]
        _, loss = model(x[:, :-1], x[:, 1:])
        model.zero_grad()
        loss.backward()
        grads = torch.cat([p.grad.view(-1) for p in params])
        dist.all_reduce(grads)
        shard.grad = grads.tensor_split(world)[rank] / world
        opt.step()
        pieces = [torch.empty(n) for n in sizes]
        dist.all_gather(pieces, shard.detach())
        new = torch.cat(pieces)
        offset = 0
```

```
        with torch.no_grad():
            for p in params:
                p.copy_(new[offset : offset + p.numel()].view_as(p))
                offset += p.numel()
```

A quarter of the optimiser state per process, the same parameters

```
ZeRO stage 1: Adam's state 1,666 KiB per process, not 6,665 KiB; parameters within 3.4e-05
```

Stage 1 cuts Adam's 8 bytes per parameter by the number of processes at the cost of one all-gather per step. Stage 3 cuts all 16 bytes and moves the parameters across the network on every forward and backward pass. PyTorch's *Fully Sharded Data Parallel* is stage 3.

Tensor parallelism

Data parallelism cannot help when one layer's matrices, or its activations, are too large for a device. *Tensor parallelism* splits the matrices themselves. In a feed-forward network, split the first layer's weights by output columns: each process computes a slice of the hidden features from the full input, with no communication. Split the second layer's weights by input rows, the same slices: each process multiplies its hidden slice by its rows and produces a partial sum of the full output. One all-reduce adds the partial sums. The SwiGLU network splits the same way, since the gate and the up projection are split identically and their product is element-wise.

A SwiGLU network split by columns and rows across processes `code/u4/parallel.py`

```
def experiment_tensor_parallel(rank: int, world: int, out: dict) -> None:
    torch.manual_seed(0)
    full = gpt.SwiGLU(TINY)
    x = torch.randn(8, TINY.d_model, requires_grad=True)
    columns = torch.arange(TINY.d_ff).tensor_split(world)[rank]
    gate, up = full.gate.weight[columns], full.up.weight[columns]
    down = full.down.weight[:, columns]
    partial = F.linear(F.silu(F.linear(x, gate)) * F.linear(x, up), down)
    total = partial.clone()
    dist.all_reduce(total)
```

The split layer equals the whole one

```
tensor-parallel SwiGLU across 4 processes: largest difference from the whole layer 2.1e-07
```

Attention splits the same way, by heads. The cost is an all-reduce of a layer's output in every forward pass and another in every backward pass, for every layer, which only fast links within one machine can sustain. Tensor parallelism is therefore used across the eight GPUs of a server, and rarely beyond.

Pipeline parallelism

Pipeline parallelism gives each device a contiguous group of layers. A batch flows through the stages in order, each sending its output activations to the next, so communication is only one activation tensor per stage boundary. The stages are often idle, because while the first stage works on a batch the second waits for it, and vice versa. Splitting each batch into m *micro-batches* lets the stages work on different micro-batches at the same time. With p stages, the pipeline needs $m + p - 1$ steps to process m micro-batches, of which each stage is busy for m, so the idle fraction, the *bubble*, is $(p - 1)/(m + p - 1)$.

A two-stage pipeline, forward only, with the batch split into micro-batches `code/u4/parallel.py`

```
for micro in (1, 2, 4, 8, 16):
    dist.barrier()
    start = time.perf_counter()
    for chunk in x.tensor_split(micro):
        if rank == 0:
            h = model.embed(chunk)
            for block in layers:
                h = block(h, cos, sin)
            dist.send(h.detach(), 1)
        else:
            h = torch.empty(len(chunk), x.shape[1], 256)
            dist.recv(h, 0)
            for block in layers:
                h = block(h, cos, sin)
    dist.barrier()
    timings[micro] = time.perf_counter() - start
```

Two stages on two processes, 64 sequences

```
pipeline of two stages, 64 sequences split into micro-batches:
  micro-batches      ms  speed-up  idle in theory
              1     179     1.00x             50%
              2     123     1.45x             33%
              4      90     1.98x             20%
              8      97     1.84x             11%
             16      94     1.90x              6%
```

With one micro-batch the stages take turns, and the pipeline is no faster than one process. Splitting the batch lets them overlap, and the time falls towards half (Figure 4.5.1). Beyond a few micro-batches the gains stop, because each micro-batch costs a message and smaller matrix multiplications run less efficiently, the trade-off measured in Chapter 1.1. Real schedules, such as 1F1B, interleave forward and backward passes of different micro-batches to limit the activations held at once, and interleaved variants that give each device several stages also shrink the bubble.

> **KEY IDEA**
> Data parallelism divides the batch and communicates gradients once per step, and ZeRO divides the stored state. Tensor parallelism divides the layers' matrices and communicates in every layer. Pipeline parallelism divides the depth and communicates activations, at the cost of a bubble. Large runs combine them, with the most communication-hungry strategy on the fastest links.

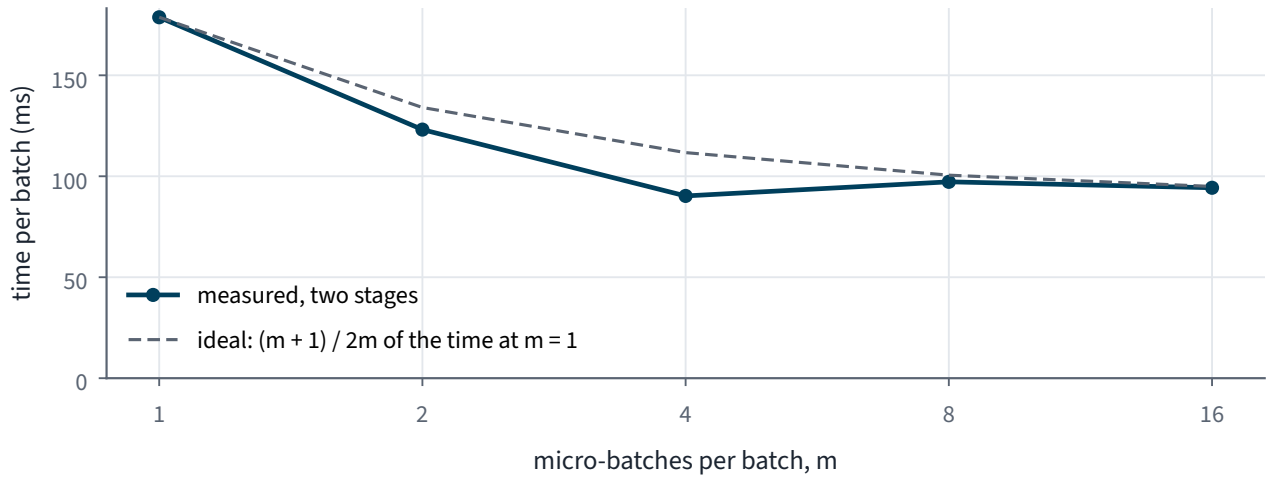


Figure 4.5.1. Time per batch against the number of micro-batches, with the theoretical bubble. Micro-batches fill the pipeline until the overhead of each message and the smaller matrices take over.

Costs at scale

For a model of P parameters in bf16 on N devices, data parallelism sends about $2 \times 2P$ bytes per device per step in its all-reduce. For 7 billion parameters that is 28 GB, about 0.3 seconds on a 100 GB/s link, every step, which is why the all-reduce is overlapped with the backward pass and starts on each layer's gradients as soon as they are ready. Tensor parallelism across t devices sends two all-reduces of each layer's activations per pass, each of $2BTd$ bytes in bf16, so with the ring's factor of two about $8BTd$ bytes per device per layer, every layer, every micro-batch. Pipeline parallelism sends one activation tensor per stage boundary per micro-batch. A typical recipe for a model of tens of billions of parameters is tensor parallelism of 8 within each server, pipeline parallelism across a few servers, and data parallelism with ZeRO across the rest, which is called *3D parallelism*.

> **IN PYTORCH**
> `torch.nn.parallel.DistributedDataParallel` implements data parallelism with bucketed, overlapped all-reduces, and `torch.distributed.fsdp.FullyShardedDataParallel` implements ZeRO stage 3. Tensor parallelism is expressed with `torch.distributed.tensor`, which describes how each tensor is split across a device mesh, and `torch.distributed.pipelining` provides pipeline schedules. Processes are started with `torchrun`, which sets the environment variables that `init_process_group` reads, as this chapter's program does by hand.

Reading

- Mohammad Shoeybi and others, "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism", 2019: tensor parallelism.
- Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase and Yuxiong He, "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models", SC 2020.
- Yanping Huang and others, "GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism", NeurIPS 2019.
- Deepak Narayanan and others, "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM", SC 2021: combining the strategies.
- Pitch Patarasuk and Xin Yuan, "Bandwidth Optimal All-reduce Algorithms for Clusters of Workstations", *Journal of Parallel and Distributed Computing* 69(2), 2009.

Labs

LAB 4.5.1 The cost of an all-reduce — 1.5 hours

Goal. Fit the latency and bandwidth of your all-reduce.

Code. `code/u4/parallel.py`

1. Time your ring and gloo's all-reduce for tensors from 4 KiB to 64 MiB, with 2, 4 and 8 processes.
2. Fit each to $\alpha + \beta S$, with S the bytes per process, and compare the fitted β with the ring's $2(N-1)/N$ factor.

DONE WHEN

- [ ] Your plots cover at least six sizes for each process count.
- [ ] You can say at what size each all-reduce becomes bandwidth-bound, from your fit.

EVIDENCE FOR THE NOTEBOOK

- The plots and the fitted constants.

LAB 4.5.2 Overlapping communication — 2 hours

Goal. Hide the all-reduce behind the backward pass.

Code. `code/u4/parallel.py`

1. Register a hook on each parameter that starts an asynchronous all-reduce, `dist.all_reduce(grad, async_op=True)`, as soon as its gradient is ready, and wait for them all before the optimiser step.
2. Measure the step time against the version that all-reduces once at the end, on a model large enough for the difference to show.

DONE WHEN

- [ ] The overlapped version gives identical parameters.
- [ ] You have both step times and can explain the difference, or its absence, on your machine.

EVIDENCE FOR THE NOTEBOOK

- The code and the timings.

LAB 4.5.3 ZeRO stage 2 — 1.5 hours

Goal. Shard the gradients as well.

Code. `code/u4/parallel.py`

1. Replace the all-reduce in `experiment_zero` with a reduce-scatter, so that each process receives only its slice of the summed gradients.
2. Measure the gradient and optimiser memory per process for 1, 2 and 4 processes.

DONE WHEN

- [ ] Parameters still match one-process training to within 10^{-4}.
- [ ] Your table shows memory falling in proportion to the number of processes.

EVIDENCE FOR THE NOTEBOOK

- The code and the table.

LAB 4.5.4 Tensor-parallel attention — 2 hours

Goal. Split attention by heads.

Code. `code/u4/parallel.py`

1. Split one of Chapter 2.4's attention layers across 2 and 4 processes, each taking whole query heads and the key-value heads they share.
2. Check the output against the whole layer, and count the bytes each process sends per forward pass.

DONE WHEN

- [ ] The split layer matches the whole one to within 10^{-5}.
- [ ] Your byte count matches the formula for one all-reduce of the layer's output.

EVIDENCE FOR THE NOTEBOOK

- The code, the check and the count.

Problem set

1. Show that a ring all-reduce over N processes sends $2(N-1)S/N$ bytes per process, and that no algorithm can send fewer than $(N-1)S/N$ in each direction.

2. A 70-billion-parameter model trains with AdamW in mixed precision, keeping float32 master weights. How many bytes of state per parameter? Spread with ZeRO stage 3 over 64 GPUs, how many GB per GPU?
3. Why can the second layer of a tensor-parallel feed-forward network not be split by columns as well? What would it cost?
4. With 4 pipeline stages, how many micro-batches keep the bubble below 10%? What does that do to each micro-batch's size for a batch of 512 sequences?
5. Data parallelism over 1,024 GPUs needs a global batch of at least 1,024 sequences. Why might that harm training, and how do large runs choose the batch size?

COMPLETION CHECKLIST

- ☐ I can explain the collectives and implement a ring all-reduce.
- ☐ I can explain data parallelism and show that it matches one-process training.
- ☐ I can explain ZeRO's three stages and what each saves and costs.
- ☐ I can split a feed-forward network or attention for tensor parallelism, and compute a pipeline's bubble.
- ☐ Lab 4.5.1–4.5.4 done-when criteria all hold, with evidence filed.

CHAPTER 4.6

An inference engine in C++

TIME	8 hours
YOU BUILD	An inference engine of about 1,100 lines of dependency-free C++20 that memory-maps an exported model, encodes text with a byte-level BPE tokeniser of its own, and generates with a KV cache, a thread pool and NEON vector code; checked against PyTorch on tokens and on logits; and timed from one thread to eight, on the course's model and on SmolLM2.
NEEDS	Chapters 2.4, 2.7 and 4.1, and C++ at the level of the Handbook's prerequisites. CMake 3.25 and Ninja.

WHY IT MATTERS

Running a trained model needs no framework. Generating a token is a sequence of matrix-vector products and a handful of small functions, and an engine that does only that can be small, fast and free of dependencies. Engines of this kind run models on laptops, phones and servers without Python. Writing one shows where the time goes when a model generates text: almost all of it is spent reading the weights from memory. That fact shapes the remaining chapters of the course, since quantisation shrinks the weights and serving reads them once for many users.

SKIP TEST

1. Why is generating one token at a time limited by memory bandwidth rather than arithmetic?
2. What does the KV cache hold for each layer and token, and why does generation need it?
3. How would you divide a matrix-vector product among eight threads, and why that way?
4. A model's weights take 540 MB. At most how many tokens per second can a machine with 100 GB/s of bandwidth generate from it, one sequence at a time?

If all four are easy, read the sections on matrix-vector products and on speed, and do Labs 4.6.3 and 4.6.4.

Core ideas

The plan

The engine needs three things from Python: the weights, the tokeniser and a reference to check itself against. `u4/export.py` writes all of them into one file: a four-byte tag, a JSON header describing the configuration, every tensor's name, kind, shape and offset, the tokeniser's tokens and merges, and the reference logits for a test prompt, then the tensors themselves, each aligned to 64 bytes.

Exporting a model, its tokeniser and a reference into one file — code/u4/export.py

```
def export(
    model: gpt.GPT, tok: bpe.BPE, path: Path, prompt: str, weights: str = "f32"
) -> None:
    c = model.config
    tensors = {"embed": model.embed.weight, "norm": model.norm.weight}
    for i, b in enumerate(model.blocks):
        tensors |= {
            f"{i}.attn_norm": b.attn_norm.weight,
            f"{i}.mlp_norm": b.mlp_norm.weight,
            f"{i}.q": b.attn.q.weight,
            f"{i}.k": b.attn.k.weight,
            f"{i}.v": b.attn.v.weight,
```

```
            f"{i}.o": b.attn.o.weight,
            f"{i}.gate": b.mlp.gate.weight,
            f"{i}.up": b.mlp.up.weight,
            f"{i}.down": b.mlp.down.weight,
        }
    if not c.tie_embeddings or weights != "f32":
        tensors["head"] = model.head.weight
    blobs, entries, offset = [], [], 0
    for name, t in tensors.items():
        array = t.detach().float().cpu().numpy()
        matrix = array.ndim == 2 and name != "embed"
        parts = {"f32": [("f32", array)]}
        if matrix and weights == "i8":
            q, s = quantise_rows_int8(array)
            parts = {"i8": [("i8", q), ("f32", s)]}
        elif matrix and weights == "q4":
            q, s = quantise_groups_int4(array)
            parts = {"q4": [("u8", q), ("f32", s)]}
        kind, pieces = next(iter(parts.items()))
        entry = {"name": name, "kind": kind, "shape": list(array.shape), "offsets": []}
        for _, piece in pieces:
            offset = (offset + ALIGN - 1) // ALIGN * ALIGN
            entry["offsets"].append(offset)
            blobs.append((offset, np.ascontiguousarray(piece).tobytes()))
            offset += piece.nbytes
        entries.append(entry)
    ids = tok.encode(prompt)
    reference_model = dequantised_copy(model, weights) if weights != "f32" else model
    with torch.no_grad():
        reference = (
            reference_model(torch.tensor([ids], device=next(model.parameters()).device))[0, -1]
            .float()
            .cpu()
        )
    header = {
        "config": {
            "vocab": c.vocab_size,
            "d_model": c.d_model,
            "layers": c.n_layers,
            "heads": c.n_heads,
            "kv_heads": c.n_kv_heads,
            "d_ff": c.d_ff,
            "max_seq": c.max_seq,
            "rope_theta": c.rope_theta,
            "norm_eps": c.norm_eps,
            "tied": c.tie_embeddings,
        },
        "tensors": entries,
        "tokenizer": {
            "tokens": [t.hex() for t in tok.token_bytes],
            "merges": [
                [a, b, m]
                for (a, b), m in sorted(tok.merged.items(), key=lambda kv: tok.rank[kv[0]])
            ],
            "special": tok.special,
            "split_digits": tok.split_digits,
        },
        "check": {"prompt": prompt, "ids": ids, "logits": reference.tolist()},
    }
    raw = json.dumps(header).encode()
    start = (8 + len(raw) + ALIGN - 1) // ALIGN * ALIGN
    with open(path, "wb") as f:
        f.write(b"AIF1" + struct.pack("<I", len(raw)) + raw)
        for at, blob in blobs:
            f.seek(start + at)
            f.write(blob)
```

The engine memory-maps the file, so loading costs nothing until the weights are used and the operating system shares and pages them as it needs. Every weight is a view into the mapping:

A weight: its kind, its shape, and pointers into the mapped file — code/engine/model.hpp

```
enum class Kind { F32, I8, Q4 };

struct Weight {
    Kind kind = Kind::F32;
    int rows = 0, cols = 0;
    const void* data = nullptr;
    const float* scale = nullptr;
    const float* f32() const { return static_cast<const float*>(data); }
};
```

Mapping the file and parsing its header — code/engine/model.cpp

```
Model::Model(const std::string& path) {
    int fd = open(path.c_str(), O_RDONLY);
    if (fd < 0) throw std::runtime_error("cannot open " + path);
    struct stat st{};
    fstat(fd, &st);
    mapped_ = static_cast<size_t>(st.st_size);
    map_ = mmap(nullptr, mapped_, PROT_READ, MAP_PRIVATE, fd, 0);
    close(fd);
    if (map_ == MAP_FAILED) throw std::runtime_error("cannot map " + path);
    const char* file = static_cast<const char*>(map_);
    if (std::memcmp(file, "AIF1", 4) != 0)
        throw std::runtime_error(path + " is not an AIF1 file");
    uint32_t length;
    std::memcpy(&length, file + 4, 4);
    header = json::parse(std::string_view(file + 8, length));
    const char* base = file + (8 + length + 63) / 64 * 64;
```

The header is JSON, read by a parser of about 170 lines in `engine/json.hpp`, so that the engine depends on nothing beyond the C++ standard library and POSIX.

The tokeniser in C++

Chapter 2.1's tokeniser in C++ needs two parts. The BPE merges are a direct translation, with the merge table in a hash map keyed by the pair of token ids:

Applying merges in rank order, as in Python — code/engine/tokenizer.cpp

```
std::vector<int> Tokenizer::encodeChunk(const std::string& chunk) const {
    std::vector<int> ids;
    for (unsigned char b : chunk) {
        if (byteId_[b] >= 0) ids.push_back(byteId_[b]);
    }
    while (ids.size() > 1) {
        int bestRank = std::numeric_limits<int>::max(), merged = -1;
        uint64_t bestKey = 0;
        for (size_t i = 0; i + 1 < ids.size(); ++i) {
            uint64_t key =
                (static_cast<uint64_t>(ids[i]) << 32) | static_cast<uint32_t>(ids[i + 1]);
            auto it = merges_.find(key);
            if (it != merges_.end() && it->second.first < bestRank) {
                bestRank = it->second.first;
                merged = it->second.second;
                bestKey = key;
            }
        }
        if (merged < 0) break;
        std::vector<int> next;
        for (size_t i = 0; i < ids.size(); ++i) {
            uint64_t key = i + 1 < ids.size() ? (static_cast<uint64_t>(ids[i]) << 32) |
                                                static_cast<uint32_t>(ids[i + 1])
                                              : 0;
            if (i + 1 < ids.size() && key == bestKey) {
                next.push_back(merged);
                ++i;
            } else {
                next.push_back(ids[i]);
```

```
            }
        }
        ids.swap(next);
    }
    return ids;
}
```

The pre-tokeniser is harder, because GPT-2's regular expression uses Unicode character classes, letters and numbers in any script, which the C++ standard library cannot match. The engine decodes UTF-8 to code points and classifies them itself, exactly for ASCII and approximately elsewhere: common spaces, punctuation, symbols and digits are recognised, and other characters count as letters. It then reproduces the expression's alternatives by hand, including the rule that a run of spaces leaves its last space for the following word:

GPT-2's pre-tokenisation, written out by hand — code/engine/tokenizer.cpp

```
void Tokenizer::pretokenise(const std::string& text, std::vector<std::string>& out) const {
    CodePoints t = decodeUtf8(text);
    std::vector<Cls> cls(t.cp.size());
    for (size_t i = 0; i < t.cp.size(); ++i) cls[i] = classify(t.cp[i]);
    auto emit = [&](size_t a, size_t b) {
        out.push_back(text.substr(t.at[a], t.at[b] - t.at[a]));
    };
    auto run = [&](size_t i, size_t end, Cls c) {
        while (i < end && cls[i] == c) ++i;
        return i;
    };
    auto segment = [&](size_t i, size_t end) {
        while (i < end) {
            if (t.cp[i] == '\'' && i + 1 < end) {
                uint32_t a = t.cp[i + 1], b = i + 2 < end ? t.cp[i + 2] : 0;
                if ((a == 'l' && b == 'l') || (a == 'v' && b == 'e') ||
                    (a == 'r' && b == 'e')) {
                    emit(i, i + 3);
                    i += 3;
                    continue;
                }
                if (a == 's' || a == 'd' || a == 'm' || a == 't') {
                    emit(i, i + 2);
                    i += 2;
                    continue;
                }
            }
            bool matched = false;
            for (Cls c : {Cls::Letter, Cls::Number, Cls::Other}) {
                size_t from = t.cp[i] == ' ' && i + 1 < end && cls[i + 1] == c ? i + 1 : i;
                if (cls[from] == c) {
                    size_t j = run(from, end, c);
                    emit(i, j);
                    i = j;
                    matched = true;
                    break;
                }
            }
            if (matched) continue;
            size_t j = run(i, end, Cls::Space);
            size_t stop = j < end && j - i >= 2 ? j - 1 : j;
            emit(i, stop);
            i = stop;
        }
    };
    size_t start = 0;
    for (size_t i = 0; i < t.cp.size(); ++i) {
        if (splitDigits_ && cls[i] == Cls::Number) {
            segment(start, i);
            emit(i, i + 1);
            start = i + 1;
        }
```

```
    }
    segment(start, t.cp.size());
}
```

For English text this gives exactly Python's tokens. For scripts and symbols outside the recognised ranges it can differ, and Lab 4.6.1 asks you to find such cases.

The forward pass

The forward pass for one token at one position mirrors Chapter 2.4's model line for line, with the keys and values of every earlier position kept in a cache:

One token through every layer, writing its keys and values into the cache — code/engine/engine.cpp

```
const float* Engine::forward(int token, int pos) {
    if (pos >= c_.max_seq)
        throw std::runtime_error("sequence longer than the model's context");
    const int d = c_.d_model, kvDim = c_.kv_heads * c_.headDim();
    std::memcpy(x_.data(), m_.embed.f32() + static_cast<size_t>(token) * d, d * sizeof(float));
    for (int l = 0; l < c_.layers; ++l) {
        const Layer& L = m_.layers[l];
        rmsnorm(xb_.data(), x_.data(), L.attn_norm.f32(), d, c_.norm_eps);
        matvec(L.q, xb_.data(), q_.data(), pool_);
        matvec(L.k, xb_.data(), k_.data(), pool_);
        matvec(L.v, xb_.data(), v_.data(), pool_);
        rope(q_.data(), c_.heads, c_.headDim(), pos, c_.rope_theta);
        rope(k_.data(), c_.kv_heads, c_.headDim(), pos, c_.rope_theta);
        std::memcpy(kAt(l, pos), k_.data(), kvDim * sizeof(float));
        std::memcpy(vAt(l, pos), v_.data(), kvDim * sizeof(float));
        attention(l, pos);
        matvec(L.o, attOut_.data(), xb2_.data(), pool_);
        for (int i = 0; i < d; ++i) x_[i] += xb2_[i];
        rmsnorm(xb_.data(), x_.data(), L.mlp_norm.f32(), d, c_.norm_eps);
        matvec(L.gate, xb_.data(), hb_.data(), pool_);
        matvec(L.up, xb_.data(), hb2_.data(), pool_);
        for (int i = 0; i < c_.d_ff; ++i)
            hb_[i] = hb_[i] / (1.0f + std::exp(-hb_[i])) * hb2_[i];
        matvec(L.down, hb_.data(), xb2_.data(), pool_);
        for (int i = 0; i < d; ++i) x_[i] += xb2_[i];
    }
    rmsnorm(x_.data(), x_.data(), m_.norm.f32(), d, c_.norm_eps);
    matvec(m_.head, x_.data(), logits_.data(), pool_);
    return logits_.data();
}
```

Rotary embeddings, in the Llama convention — code/engine/ops.cpp

```
void rope(float* v, int heads, int headDim, int pos, float theta) {
    int half = headDim / 2;
    for (int i = 0; i < half; ++i) {
        float freq = 1.0f / std::pow(theta, static_cast<float>(2 * i) / headDim);
        float angle = static_cast<float>(pos) * freq;
        float c = std::cos(angle), s = std::sin(angle);
        for (int h = 0; h < heads; ++h) {
            float* p = v + h * headDim;
            float x1 = p[i], x2 = p[i + half];
            p[i] = x1 * c - x2 * s;
            p[i + half] = x2 * c + x1 * s;
        }
    }
}
```

Attention over the cache, one head per task — code/engine/engine.cpp

```
void Engine::attention(int layer, int pos) {
    const int hd = c_.headDim(), group = c_.heads / c_.kv_heads;
    const float scale = 1.0f / std::sqrt(static_cast<float>(hd));
    pool_.parallelFor(c_.heads, [&](int begin, int end) {
        for (int h = begin; h < end; ++h) {
            const float* q = q_.data() + h * hd;
            float* scores = att_.data() + static_cast<size_t>(h) * c_.max_seq;
            int kv = h / group;
            for (int t = 0; t ⩽ pos; ++t) {
                const float* k = kAt(layer, t) + kv * hd;
                float dot = 0.0f;
                for (int i = 0; i < hd; ++i) dot += q[i] * k[i];
                scores[t] = dot * scale;
            }
            softmax(scores, pos + 1);
            float* out = attOut_.data() + h * hd;
            std::fill(out, out + hd, 0.0f);
            for (int t = 0; t ⩽ pos; ++t) {
                const float* v = vAt(layer, t) + kv * hd;
                for (int i = 0; i < hd; ++i) out[i] += scores[t] * v[i];
            }
        }
    });
}
```

Generation is then a loop: run the prompt through `forward`, sample a token from the last logits, feed it back, and repeat until a stop token or the context is full.

Greedy decoding, or top-p sampling at a temperature — code/engine/engine.cpp

```
int Engine::sample(const float* logits, const Sampling& s) {
    if (s.temperature ⩽ 0.0f)
        return static_cast<int>(std::max_element(logits, logits + c_.vocab) - logits);
    std::vector<float> p(logits, logits + c_.vocab);
    for (float& v : p) v /= s.temperature;
    softmax(p.data(), c_.vocab);
    std::vector<int> order(c_.vocab);
    std::iota(order.begin(), order.end(), 0);
    std::sort(order.begin(), order.end(), [&](int a, int b) { return p[a] > p[b]; });
    float kept = 0.0f;
    size_t n = 0;
    while (n < order.size() && kept < s.top_p) kept += p[order[n++]];
    std::uniform_real_distribution<float> uniform(0.0f, kept);
    float r = uniform(rng_);
    for (size_t i = 0; i < n; ++i) {
        r -= p[order[i]];
        if (r ⩽ 0.0f) return order[i];
    }
    return order[n - 1];
}
```

Matrix-vector products

During generation each projection multiplies a matrix by a single vector, and nearly all the time goes there. Each output element is the dot product of one weight row with the input, so the rows divide naturally among threads, with no two threads writing the same output:

A matrix-vector product, rows shared among threads, for each kind of weight — code/engine/ops.cpp

```
void matvec(const Weight& w, const float* x, float* y, ThreadPool& pool) {
    pool.parallelFor(w.rows, [&](int begin, int end) {
        for (int r = begin; r < end; ++r) {
            size_t row = static_cast<size_t>(r) * w.cols;
            if (w.kind == Kind::F32) {
                y[r] = dotF32(w.f32() + row, x, w.cols);
            } else if (w.kind == Kind::I8) {
                y[r] = dotI8(static_cast<const int8_t*>(w.data) + row, x, w.cols) * w.scale[r];
            } else {
                y[r] = dotQ4(static_cast<const uint8_t*>(w.data) + row / 2, w.scale + row / 32,
                             x, w.cols);
            }
        }
    });
}
```

The thread pool keeps its workers alive between calls and hands out chunks of rows through an atomic counter. A generated token makes about sixty such calls for the course's model, so the cost of waking threads matters. The workers therefore spin, checking a counter, instead of sleeping on a condition variable, which would add tens of microseconds per call.

The thread pool — code/engine/ops.hpp

```
class ThreadPool {
public:
    explicit ThreadPool(int threads);
    ~ThreadPool();
    void parallelFor(int n, const std::function<void(int, int)>& fn);
    int size() const { return static_cast<int>(workers_.size()) + 1; }

private:
    std::vector<std::thread> workers_;
    const std::function<void(int, int)>* job_ = nullptr;
    int n_ = 0, chunk_ = 1;
    std::atomic<int> next_{0}, pending_{0};
    std::atomic<uint64_t> epoch_{0};
    std::atomic<bool> stop_{false};
    void work();
};
```

Publishing a job, working on it too, and waiting for the workers — code/engine/ops.cpp

```
void ThreadPool::parallelFor(int n, const std::function<void(int, int)>& fn) {
    if (workers_.empty()) {
        fn(0, n);
        return;
    }
    job_ = &fn;
    n_ = n;
    chunk_ = std::max(1, n / (size() * 8));
    next_.store(0, std::memory_order_relaxed);
    pending_.store(static_cast<int>(workers_.size()), std::memory_order_relaxed);
    epoch_.fetch_add(1, std::memory_order_release);
    work();
    while (pending_.load(std::memory_order_acquire) > 0) std::this_thread::yield();
}
```

The dot product itself uses the CPU's vector unit, NEON on ARM, four lanes of float32 per instruction. Four independent accumulators let four multiply-adds be in flight at once. With one accumulator, each would wait for the previous one to finish, and the loop would run at a quarter of the speed.

A NEON dot product with four accumulators, and a scalar fallback — code/engine/ops.cpp

```
float dotF32(const float* w, const float* x, int n) {
    int i = 0;
    float sum = 0.0f;
#if defined(__ARM_NEON)
    float32x4_t a0 = vdupq_n_f32(0), a1 = a0, a2 = a0, a3 = a0;
    for (; i + 16 <= n; i += 16) {
        a0 = vfmaq_f32(a0, vld1q_f32(w + i), vld1q_f32(x + i));
        a1 = vfmaq_f32(a1, vld1q_f32(w + i + 4), vld1q_f32(x + i + 4));
        a2 = vfmaq_f32(a2, vld1q_f32(w + i + 8), vld1q_f32(x + i + 8));
        a3 = vfmaq_f32(a3, vld1q_f32(w + i + 12), vld1q_f32(x + i + 12));
    }
    sum = vaddvq_f32(vaddq_f32(vaddq_f32(a0, a1), vaddq_f32(a2, a3)));
#endif
    for (; i < n; ++i) sum += w[i] * x[i];
    return sum;
}
```

Checking against PyTorch

Encoding the reference prompt and comparing the logits — code/engine/main.cpp

```
int check(const aif::Model& model, aif::Tokenizer& tok, aif::Engine& engine) {
    const auto& ref = model.header["check"];
    std::vector<int> want;
    for (const auto& v : ref["ids"].array()) want.push_back(static_cast<int>(v.num()));
    std::vector<int> got = tok.encode(ref["prompt"].str());
    if (got != want) {
        std::printf("CHECK FAILED  the prompt's tokens differ from Python's\n");
        return 1;
    }
    const float* logits = nullptr;
    for (size_t i = 0; i < got.size(); ++i)
        logits = engine.forward(got[i], static_cast<int>(i));
    double worst = 0.0;
    const auto& expected = ref["logits"].array();
    for (size_t i = 0; i < expected.size(); ++i)
        worst = std::max(worst, std::fabs(logits[i] - expected[i].num()));
    std::printf(
        "prompt of %zu tokens: tokens identical to Python's; largest logit difference from "
        "PyTorch %.2e\n",
        got.size(), worst);
    return worst < 1e-3 ? 0 : 1;
}
```

The engine against PyTorch, on the course's model

```
$ ./build/bin/aif_engine runs/engine/tinystories-28m-f32.bin --check
runs/engine/tinystories-28m-f32.bin: 8 layers, d_model 512, vocabulary 8192, 106.0 MiB of
    weights, 12 threads
prompt of 11 tokens: tokens identical to Python's; largest logit difference from PyTorch
    1.00e-05
```

The tokens are identical and the logits agree to about 10^{-5}, the rounding of float32 arithmetic done in a different order. The engine then generates a story from the same model:

Chapter 2.5's model, running in C++

```
$ ./build/bin/aif_engine runs/engine/tinystories-28m-f32.bin --prompt 'Once upon a time, there
was a little dog named' --tokens 120
runs/engine/tinystories-28m-f32.bin: 8 layers, d_model 512, vocabulary 8192, 106.0 MiB of
    weights, 12 threads
Once upon a time, there was a little dog named Spot. Spot loved to play with his ball. One
    day, Spot saw a big cat. The cat was very rude. Spot wanted to play with the cat, but
    the cat did not want to play.
Spot had an idea. He would play with the cat and his ball. He would throw the ball and the
    cat would run to catch it. Spot was very happy. He played with the cat and his ball all
    day long.
But then, something unexpected happened. The cat was not rude anymore. The cat was nice and
    played with Spot and his ball. They had lots of
```

Where the time goes

Generating one token reads every weight once and uses it in one multiplication and one addition, two FLOPs per four bytes. That is an arithmetic intensity of 0.5, far left of the ridge of any processor's roofline. Decoding one sequence is therefore limited by memory bandwidth, and the best possible speed is the bandwidth divided by the bytes of weights read per token.

Tokens per second by threads, three models, float32 weights, on the M2 Pro's CPU

```
$ uv run -m u4.engine_bench
tokens per second generating 200 tokens, by threads, on the CPU:
model            weights     MiB      1      2      4      8     12
tinystories-28m  f32         106    386    564    741    762    495
SmolLM2-135M     f32         516     85    114    144    144    100
SmolLM2-360M     f32        1383     33     50     62     63     57
```

One thread cannot keep the memory system busy, because a core can have only so many requests outstanding at a time. More threads raise the speed up to four, and beyond that it barely moves. At their best the three models read their weights at similar rates, between 78 and 91 GB/s, which suggests that all three meet the same limit, the bandwidth that the CPU's cores can draw from memory. Lab 4.6.3 measures that limit directly. Twelve threads, one for every core, are slower than eight for every model. The likely cause is that with every core busy, anything else the machine runs preempts one of the engine's threads, and each operation then waits for its slowest thread. From here, generation gets faster only by reading fewer bytes per token, the subject of Chapter 4.7, or by reading them once for many sequences, the subject of Chapter 4.8.

> **KEY IDEA**
> Generating a token reads every weight once, so single-sequence decoding is bound by memory bandwidth: its speed is at most the bandwidth divided by the model's size in bytes. Threads and vector instructions can only bring the engine closer to that bound.

> **IN PRACTICE**
> `llama.cpp`, built on the `ggml` library, is the best-known engine of this kind, with C and C++ code, memory-mapped weights in its GGUF format, hand-written kernels for every CPU and GPU it supports, and quantised weights of about 1.5 to 8 bits. Andrej Karpathy's `llama2.c` is a single-file engine much like this chapter's. PyTorch's own route to devices without Python is ExecuTorch, which compiles a model ahead of time.

Reading

- Reiner Pope and others, "Efficiently Scaling Transformer Inference", MLSys 2023: the arithmetic of inference.
- Andrej Karpathy, `llama2.c`, 2023, a minimal inference engine in C.

- Georgi Gerganov and contributors, `llama.cpp` and `ggml`, in particular the CPU matrix-vector kernels.
- Arm, *Neon Intrinsics Reference*, for the vector instructions used here.

Labs

LAB 4.6.1 Your model, in C++ 1.5 hours

Goal. Run your own models through the engine and find the tokeniser's limits.

Code. `code/engine`

1. Build the engine with CMake, export your Chapter 2.5 model and SmolLM2-135M-Instruct, and run `--check` on both.
2. Generate a story from your model and an answer from SmolLM2 with its chat template written into the prompt.
3. Find three strings, in other scripts or with unusual symbols, on which the engine's tokens differ from Python's, and explain each.

DONE WHEN

- [ ] Both checks pass.
- [ ] Your three strings come with the classification error that causes each.

EVIDENCE FOR THE NOTEBOOK

- The outputs and the three explanations.

LAB 4.6.2 Batched prefill 2 hours

Goal. Process the prompt with matrix-matrix products.

Code. `code/engine/engine.cpp`

1. Add `prefill(tokens)` that runs all the prompt's tokens through each layer together, so that each projection becomes a matrix-matrix product, with causal attention among the prompt's positions.
2. Check that the logits after prefill equal those after feeding the prompt one token at a time.
3. Measure prefill time for prompts of 16, 64 and 256 tokens both ways.

DONE WHEN

- [ ] The logits agree to within 10^{-4}.
- [ ] Your table shows the speed-up growing with prompt length, and you can explain why from arithmetic intensity.

EVIDENCE FOR THE NOTEBOOK

- The code, the check and the table.

LAB 4.6.3 Against the bound 1.5 hours

Goal. Measure how close the engine comes to the memory bound.

Code. `code/engine/ops.cpp`

1. Measure your CPU's read bandwidth with a simple program that sums a 1 GiB array with 1 to 12 threads.
2. Compute each model's bound in tokens per second and compare with the engine's best.
3. Find the share of each token's time spent outside matrix-vector products, by timing them separately.

DONE WHEN
- [ ] You have the bandwidth curve, the bounds and the measured speeds.
- [ ] You can account for any gap between each model and its bound.

EVIDENCE FOR THE NOTEBOOK
- The measurements and the accounting.

LAB 4.6.4 Long contexts — 2 hours

Goal. Make attention scale with the context.

Code. `code/engine/engine.cpp`

1. Measure milliseconds per token at positions 100, 500 and 2,000 of a long generation with SmolLM2-135M.
2. Replace the attention's scalar loops with the NEON dot product and a vectorised weighted sum.

DONE WHEN
- [ ] Your measurements show attention's growing share of time before the change.
- [ ] After the change the logits still match PyTorch, and the time per token at position 2,000 has fallen.

EVIDENCE FOR THE NOTEBOOK
- Both sets of timings and the check.

Problem set

1. How many bytes does the course's model read per generated token in float32, counting the weights and the KV cache at position 500?
2. Why is the output layer of SmolLM2-135M a large share of its decoding time? What fraction of its weights is it?
3. The thread pool's workers spin while waiting. What does that cost when the engine is idle, and how could it sleep without slowing generation?
4. Why must the engine's rotary embeddings use the same pairing of dimensions as the PyTorch model, and what would happen if they did not?
5. With a memory-mapped file of weights larger than the machine's free memory, what does the operating system do during generation, and how fast would it be?

COMPLETION CHECKLIST
- [ ] I can describe each step of generating a token and implement it without a framework.
- [ ] I can implement and test a tokeniser and a forward pass against a reference.
- [ ] I can parallelise and vectorise a matrix-vector product, and explain the role of independent accumulators.
- [ ] I can compute the memory bound on decoding speed and compare an engine against it.
- [ ] Lab 4.6.1–4.6.4 done-when criteria all hold, with evidence filed.

CHAPTER 4.7

Quantisation

TIME	6 hours
YOU BUILD	Weights rounded to 8, 4 and 3 bits in groups of several sizes, with the cost in validation loss measured on the course's model and on SmolLM2; GPTQ, written from its definition, which corrects each column's rounding error using the next columns; a KV cache in 8 bits; and the engine's 8- and 4-bit kernels, checked against PyTorch and timed.
NEEDS	Chapters 4.3 and 4.6.

WHY IT MATTERS

Chapter 4.6 showed that generating text is limited by the bytes of weights read per token. Storing each weight in 8 bits instead of 32 reads a quarter of the bytes, and in 4 bits an eighth. If the model's predictions barely change, that is a large speed-up at almost no cost, and it also lets a model fit in a phone's memory. This chapter measures how far the bits can be cut before the predictions suffer, and how a better rounding method, GPTQ, pushes that limit further.

SKIP TEST

1. What is symmetric absmax quantisation to b bits, and what does the scale store?
2. Why do quantised weights use a scale per group of 32 or 128 values rather than one per tensor?
3. What does GPTQ do differently from rounding each weight to the nearest level?
4. Why can 4-bit weights be slower than 8-bit weights on a CPU, despite moving half the bytes?

If all four are easy, read the sections on GPTQ and on speed, and do Labs 4.7.2 and 4.7.4.

Core ideas

Rounding weights

Symmetric absmax quantisation maps a group of weights to integers: divide by a scale chosen so that the largest magnitude maps to the largest level, $2^{b-1} - 1$, round to the nearest integer, and store the integers with the scale. Dequantisation multiplies the integers by the scale. The rounding error of each weight is at most half a step, and the step is the group's largest magnitude divided by $2^{b-1} - 1$, so a single large weight coarsens the whole group. Scales are therefore kept per group: one per row of a matrix for 8 bits, and one per 32 to 128 consecutive weights for 4 bits. Each scale costs 32 bits, a quarter of a bit per weight with groups of 128.

Round-to-nearest quantisation to any width, with a scale per group — code/u4/quantise.py

```
def round_to_nearest(w: torch.Tensor, bits: int, group: int) -> torch.Tensor:
    top = 2 ** (bits - 1) - 1
    rows, cols = w.shape
    g = F.pad(w, (0, -cols % group)).view(rows, -1, group)
    scale = g.abs().amax(-1, keepdim=True).clamp(min=1e-8) / top
    q = torch.clamp(torch.round(g / scale), -top - 1, top) * scale
    return q.view(rows, -1)[:, :cols]
```

The program quantises every matrix of every block of a model, leaves the embedding and norms alone, and measures the validation loss:

Five quantisations of the same model — code/u4/quantise.py

```
variants = [
    ("int8, one scale per row", 8, None),
    ("int4, groups of 128", 4, 128),
    ("int4, groups of 32", 4, 32),
    ("int3, groups of 32", 3, 32),
    ("int4 GPTQ, groups of 128", 4, 128),
]
for name, bits, group in variants:
    q = copy.deepcopy(model)
    if "GPTQ" in name:
        calibrate_and_quantise(q, calib, bits, group)
    else:
        for layer in linears(q):
            w = layer.weight.data
            layer.weight.data = round_to_nearest(w, bits, group or w.shape[1])
    loss = train.evaluate(q, valid)
```

The cost of fewer bits on the course's model

```
$ uv run -m u4.quantise
tinystories-28m: 23.6M parameters in its block matrices; validation loss 1.2138 in float32
weights                        bits/weight  block MiB     loss   change
float32                              32.00       90.0   1.2138  +0.0000
int8, one scale per row               8.05       22.6   1.2138  -0.0001
int4, groups of 128                   4.25       12.0   1.2315  +0.0177
int4, groups of 32                    5.00       14.1   1.2256  +0.0117
int3, groups of 32                    4.00       11.2   1.2802  +0.0664
int4 GPTQ, groups of 128              4.25       12.0   1.2235  +0.0096
KV cache in int8, one scale per head and position: second-half loss 1.1555 → 1.1555
```

The same on SmolLM2-135M, on TinyStories' validation text

```
$ uv run -m u4.quantise --model HuggingFaceTB/SmolLM2-135M
HuggingFaceTB/SmolLM2-135M: 106.2M parameters in its block matrices; validation loss 2.0336
    in float32
weights                        bits/weight  block MiB     loss   change
float32                              32.00      405.0   2.0336  +0.0000
int8, one scale per row               8.05      101.8   2.0352  +0.0016
int4, groups of 128                   4.27       54.1   2.3881  +0.3545
int4, groups of 32                    5.00       63.3   2.2799  +0.2464
int3, groups of 32                    4.00       50.6   3.5964  +1.5628
int4 GPTQ, groups of 128              4.27       54.1   2.1565  +0.1229
KV cache in int8, one scale per head and position: second-half loss 2.0150 → 2.0149
```

On the course's model, eight bits with a scale per row cost nothing measurable. Four bits cost one to two hundredths of a nat, less with smaller groups at the price of more scales, and three bits nearly seven hundredths. SmolLM2-135M is far more sensitive. Eight bits cost two thousandths of a nat, four bits from a quarter to over a third of a nat, and three bits more than one and a half, which makes the model unusable. SmolLM2 was trained on over a thousand times as many tokens per parameter, and studies of quantisation across many training runs find that the more data a model has absorbed, the more rounding its weights costs it. The general pattern holds on both models: 8-bit weights are effectively free, 4-bit weights are the usual compromise and need care with well-trained models, and below 4 bits quality falls quickly without better methods.

GPTQ

Rounding each weight to its nearest level ignores how weights combine. A layer computes $y = Wx$, and what matters is the error in y over the inputs the layer actually sees. GPTQ quantises the columns of W one at a time, and after rounding a column, adjusts the columns not yet quantised to cancel the error it introduced, as far as the inputs allow.

DERIVATION

For a row w of W and inputs x collected from calibration text, the expected squared output error of a change δ is half of $\delta^T H \delta$, with $H = 2E\left[xx^T\right]$. Quantising coordinate q changes it by $\delta_q = \text{quant}\left(w_q\right) - w_q$. The change to the remaining coordinates that minimises the error given that one is $\delta_q H^{-1}_{:,q} / H^{-1}_{qq}$, the optimal-brain-surgeon update. GPTQ applies it to all rows at once, column by column, using the Cholesky factor of H^{-1} so that the whole update costs about as much as one matrix multiplication per layer.

GPTQ for one layer: quantise a column, push its error onto the columns that remain — code/u4/quantise.py

```
def gptq(
    w: torch.Tensor, x: torch.Tensor, bits: int, group: int, damp: float = 0.01
) -> torch.Tensor:
    """Quantise columns in turn, pushing each one's error onto those not yet done."""
    device = w.device
    w, x = w.cpu().double().clone(), x.cpu().double()  # MPS has no float64
    h = 2 * x.T @ x / len(x)
    h += damp * h.diagonal().mean() * torch.eye(len(h), dtype=h.dtype, device=h.device)
    inverse = torch.linalg.cholesky(
        torch.cholesky_inverse(torch.linalg.cholesky(h)), upper=True
    )
    top = 2 ** (bits - 1) - 1
    out = torch.zeros_like(w)
    for j in range(w.shape[1]):
        if j % group == 0:
            scale = w[:, j : j + group].abs().amax(1).clamp(min=1e-8) / top
        q = torch.clamp(torch.round(w[:, j] / scale), -top - 1, top) * scale
        out[:, j] = q
        error = (w[:, j] - q) / inverse[j, j]
        w[:, j + 1 :] -= error[:, None] * inverse[j, j + 1 :][None, :]
    return out.to(device, torch.float32)
```

Each layer's inputs come from running 32 windows of training text through the model, with the layers before it already quantised, so that each layer's correction also compensates for the errors of the layers before it. The GPTQ row of each model's console shows the result. With the same groups of 128 and the same number of bits as plain rounding, GPTQ's loss is closer to float32's: it removes nearly half of the damage of 4 bits on the course's model, and two-thirds on SmolLM2, where there was more to remove.

Quantising the KV cache

At long contexts and large batches, Chapter 2.7's KV cache rivals the weights in size. It also quantises well. Storing each key and value vector in 8 bits with its own scale changes the loss by at most a ten-thousandth of a nat on either model.

Every key and value rounded to 8 bits as it enters the cache — code/u4/quantise.py

```
def quantised_kv(cache_update):
    def update(self, layer, k, v):
        def roundtrip(t):
            scale = t.abs().amax(-1, keepdim=True).clamp(min=1e-8) / 127
            return torch.round(t / scale).clamp(-127, 127) * scale

        return cache_update(self, layer, roundtrip(k), roundtrip(v))

    return update
```

The engine's quantised kernels

Chapter 4.6's engine reads three kinds of weight. 8-bit rows store one byte per weight and a float32 scale per row, and 4-bit groups pack two weights per byte with a float32 scale per 32 weights. The dot products convert the integers to floats in vector registers and multiply as before:

Dot products with 8-bit and 4-bit weights — code/engine/ops.cpp

```
float dotI8(const int8_t* w, const float* x, int n) {
    int i = 0;
    float sum = 0.0f;
#if defined(__ARM_NEON)
    float32x4_t a0 = vdupq_n_f32(0), a1 = a0, a2 = a0, a3 = a0;
    for (; i + 16 <= n; i += 16) {
        int8x16_t bytes = vld1q_s8(w + i);
        int16x8_t low = vmovl_s8(vget_low_s8(bytes)), high = vmovl_s8(vget_high_s8(bytes));
        a0 = vfmaq_f32(a0, vcvtq_f32_s32(vmovl_s16(vget_low_s16(low))), vld1q_f32(x + i));
        a1 = vfmaq_f32(a1, vcvtq_f32_s32(vmovl_s16(vget_high_s16(low))), vld1q_f32(x + i + 4));
        a2 = vfmaq_f32(a2, vcvtq_f32_s32(vmovl_s16(vget_low_s16(high))), vld1q_f32(x + i + 8));
        a3 = vfmaq_f32(a3, vcvtq_f32_s32(vmovl_s16(vget_high_s16(high))),
                       vld1q_f32(x + i + 12));
    }
    sum = vaddvq_f32(vaddq_f32(vaddq_f32(a0, a1), vaddq_f32(a2, a3)));
#endif
    for (; i < n; ++i) sum += static_cast<float>(w[i]) * x[i];
    return sum;
}

float dotQ4(const uint8_t* w, const float* scale, const float* x, int n) {
    float sum = 0.0f;
    for (int g = 0; g < n / 32; ++g) {
        const uint8_t* p = w + g * 16;
        const float* xg = x + g * 32;
        float part = 0.0f;
#if defined(__ARM_NEON)
        uint8x16_t bytes = vld1q_u8(p);
        int8x16_t eight = vdupq_n_s8(8);
        int8x16_t lo = vsubq_s8(
            veorq_s8(vreinterpretq_s8_u8(vandq_u8(bytes, vdupq_n_u8(0xF))), eight), eight);
        int8x16_t hi =
            vsubq_s8(veorq_s8(vreinterpretq_s8_u8(vshrq_n_u8(bytes, 4)), eight), eight);
        int8x16x2_t values = vzipq_s8(lo, hi);
        float32x4_t acc = vdupq_n_f32(0);
        for (int half = 0; half < 2; ++half) {
            int16x8_t a = vmovl_s8(vget_low_s8(values.val[half]));
            int16x8_t b = vmovl_s8(vget_high_s8(values.val[half]));
            const float* xs = xg + half * 16;
            acc = vfmaq_f32(acc, vcvtq_f32_s32(vmovl_s16(vget_low_s16(a))), vld1q_f32(xs));
            acc =
                vfmaq_f32(acc, vcvtq_f32_s32(vmovl_s16(vget_high_s16(a))), vld1q_f32(xs + 4));
            acc = vfmaq_f32(acc, vcvtq_f32_s32(vmovl_s16(vget_low_s16(b))), vld1q_f32(xs + 8));
            acc =
                vfmaq_f32(acc, vcvtq_f32_s32(vmovl_s16(vget_high_s16(b))), vld1q_f32(xs + 12));
        }
        sum += vaddvq_f32(acc) * scale[g];
        continue;
#endif
        for (int j = 0; j < 16; ++j) {
            int lo = (p[j] & 0xF) ^ 8, hi = (p[j] >> 4) ^ 8;
            part += static_cast<float>(lo - 8) * xg[2 * j] +
                    static_cast<float>(hi - 8) * xg[2 * j + 1];
        }
        sum += part * scale[g];
    }
    return sum;
}
```

The export for each format computes its reference logits with the same rounding, so the engine's check tests its arithmetic exactly:

The quantised engines against PyTorch running the same rounded weights, and their speed on the M2 Pro's CPU

```
$ uv run -m u4.engine_bench --weights f32 i8 q4 --threads 1 8 --check
tokens per second generating 200 tokens, by threads, on the CPU:
model            weights     MiB      1      8   logit difference
tinystories-28m  f32         106    414    761   1.0e-05
tinystories-28m  i8           43    437    941   1.2e-05
tinystories-28m  q4           33    354   1540   1.1e-05
SmolLM2-135M     f32         516     83    132   6.3e-05
SmolLM2-135M     i8          240     90    236   6.5e-05
SmolLM2-135M     q4          191     73    232   5.1e-05
SmolLM2-360M     f32        1383     32     62   7.7e-05
SmolLM2-360M     i8          529     35    131   5.1e-05
SmolLM2-360M     q4          399     28    145   5.0e-05
```

With eight threads, 8-bit weights roughly double SmolLM2's speed, in line with the bytes they save. 4-bit weights cut the bytes by a further fifth to a quarter, but their dot product does more work per weight, unpacking two values from each byte and correcting their signs, and on the CPU that work can cost as much as the bytes it saves. Whether 4 bits are faster than 8 depends on whether the kernel is bound by memory or by arithmetic, which depends on the machine and the threads, the trade-off of Chapter 4.1. Production engines use integer dot-product instructions on quantised activations as well, which removes the per-weight conversions.

KEY IDEA

Quantisation reduces the bits per weight at the cost of error in the model's outputs. 8 bits with a scale per row are effectively free. 4 bits with group scales cost a little or a lot, depending on how much the model has learned, and GPTQ recovers much of that cost by correcting each rounding error with the weights that follow. Whether fewer bits run faster depends on whether the kernel was limited by memory or by arithmetic.

IN PYTORCH

`torch.ao.quantization` and the `torchao` library provide int8 and int4 weight quantisation for PyTorch models, with fused kernels on supported hardware. In the wider ecosystem, GPTQ and AWQ checkpoints load through Hugging Face's integrations, and `llama.cpp`'s formats, such as Q4_K and Q8_0, are block quantisations of this chapter's kind with refinements.

Reading

- Elias Frantar, Saleh Ashkboos, Torsten Hoefler and Dan Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers", ICLR 2023.
- Tim Dettmers, Mike Lewis, Younes Belkada and Luke Zettlemoyer, "LLM.int8()", NeurIPS 2022: outliers and mixed-precision decomposition.
- Ji Lin and others, "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration", MLSys 2024.
- Babak Hassibi and David Stork, "Second Order Derivatives for Network Pruning: Optimal Brain Surgeon", NeurIPS 1992: the update GPTQ builds on.
- Tanishq Kumar and others, "Scaling Laws for Precision", ICLR 2025: why models trained on more data lose more to quantisation.

Labs

LAB 4.7.1 The bits curve — 1 hour

Goal. Map loss against bits per weight.

Code. `code/u4/quantise.py`

1. Quantise your Chapter 2.5 model with round-to-nearest at 8, 6, 5, 4, 3 and 2 bits, with groups of 32 and of 128.
2. Plot the change in validation loss against bits per weight, including the scales.

DONE WHEN

- [ ] Your plot has twelve points.
- [ ] You can name the cheapest setting whose loss is within 0.01 nats of float32.

EVIDENCE FOR THE NOTEBOOK

- The plot and the choice.

LAB 4.7.2 Outliers — 1.5 hours

Goal. Find the weights that make quantisation hard.

Code. `code/u4/quantise.py`

1. For each matrix of SmolLM2-135M, compute the ratio of its largest magnitude to its standard deviation, per row and per group of 128.
2. Quantise the ten matrices with the largest ratios to 4 bits and the rest to 8, and the reverse, and compare the losses.

DONE WHEN

- [ ] You have the ratios for every matrix and the two losses.
- [ ] You can say which layers are most sensitive and give a reason based on the ratios.

EVIDENCE FOR THE NOTEBOOK

- The table of ratios and the comparison.

LAB 4.7.3 GPTQ, examined — 1.5 hours

Goal. See what GPTQ's corrections do.

Code. `code/u4/quantise.py`

1. For one layer, compute the output error on calibration inputs after round-to-nearest and after GPTQ, both at 4 bits with groups of 128.
2. Vary the number of calibration windows from 4 to 64 and the damping from 0.001 to 0.1, and record the validation loss.

DONE WHEN

- [ ] GPTQ's output error is smaller than rounding's on the calibration inputs, and you have the ratio.
- [ ] Your table shows how much calibration data GPTQ needs on this model.

EVIDENCE FOR THE NOTEBOOK

- The errors and the table.

LAB 4.7.4 Faster 4-bit kernels — 2 hours

Goal. Make the engine's 4-bit path beat its 8-bit path.

Code. `code/engine/ops.cpp`

1. Profile the engine's 4-bit dot product and find its most expensive instructions.

2. Quantise the input vector to 8 bits per group as well, and compute the dot products with integer multiply-accumulate instructions, such as NEON's `vdotq_s32`, converting to float once per group.

DONE WHEN

- [ ] Your kernel's logits match the reference within 10^{-2}; raise the tolerance of `--check` from 10^{-3} for this, since the input vector is now rounded too.
- [ ] Your 4-bit engine generates faster than the 8-bit one on SmolLM2-135M at your best thread count.

EVIDENCE FOR THE NOTEBOOK

- The code, the check and the timings.

Problem set

1. A group of 128 weights has one weight of 2.0 and the rest below 0.1. At 4 bits with absmax scaling, what is the step size, and how many of the small weights round to zero?
2. Count the bits per weight of 4-bit groups of 32 with a float16 scale per group, and with a float32 scale.
3. Why does GPTQ quantise layers in order and collect each layer's inputs from the partly quantised model?
4. The KV cache of a 70B model at 128K tokens is 40 GiB in bfloat16. What is it in 8 bits with a float16 scale per vector of 128?
5. Explain why quantising weights to 4 bits speeds up decoding one sequence, but barely speeds up training or prefilling a long prompt.

COMPLETION CHECKLIST

- [ ] I can quantise weights with absmax scaling and explain the role of group size.
- [ ] I can measure the cost of quantisation in validation loss and choose a format.
- [ ] I can explain GPTQ's column-by-column error correction.
- [ ] I can explain when fewer bits per weight make a kernel faster, and when they do not.
- [ ] Lab 4.7.1–4.7.4 done-when criteria all hold, with evidence filed.

CHAPTER 4.8

Serving

TIME	6 hours
YOU BUILD	A paged KV cache that allocates memory in blocks; batched decoding of sequences of different lengths in one forward pass; a scheduler that admits requests as others finish; one load of chat requests served one at a time, in static batches and with continuous batching, measured for throughput and latency; speculative decoding, a small model drafting for a large one, checked to give exactly the large model's output; and a streaming HTTP endpoint.
NEEDS	Chapters 2.7, 4.1 and 4.6.

WHY IT MATTERS

A deployed model serves many people at once. Its users experience its latency, the time until the first word and the speed at which the rest arrive, while its running cost depends on its throughput in tokens per second per machine. Decoding one sequence at a time leaves the GPU almost idle, waiting on memory, as Chapter 2.7 showed. This capstone builds the three techniques that make serving efficient: batching many sequences through each pass over the weights, a cache layout that lets sequences of any length share memory, and speculative decoding, which spends cheap computation to save expensive memory traffic.

SKIP TEST

1. Why does batching decode steps raise throughput almost in proportion to the batch, until it does not?
2. What problem does a paged KV cache solve, and what does it cost?
3. What is the difference between static and continuous batching, and which latency does each hurt?
4. How does speculative decoding produce exactly the large model's greedy output while running the large model fewer times?

If all four are easy, read the sections on the scheduler and on speculative decoding, and do Labs 4.8.2 and 4.8.4.

Core ideas

What to measure

A request's *time to first token* is spent waiting for a slot and processing its prompt; its *inter-token latency* is the time per generated token after that. The server's *throughput* is generated tokens per second over all requests. These measures conflict. A server that runs one request at a time gives that request the fastest tokens and everyone else a long wait, while large batches raise throughput and slow each step. Designing a server means choosing a point on that trade-off, measured under a realistic load of requests arriving at random, with prompts and replies of varied lengths.

Batching decode steps

Generating one token reads every weight once, an intensity near 1 FLOP per byte with 16-bit weights. Generating one token for each of N sequences in the same pass reads the weights once for all of them, so the intensity grows with N and the step's time barely changes until the arithmetic catches up with the

memory. Throughput rises almost N-fold. Two things complicate batching: the sequences have different lengths, so their attention covers different numbers of keys, and they start and finish at different times.

A paged KV cache

Chapter 2.7's cache gave each sequence one contiguous tensor. A server holding many sequences of different lengths, which start and finish at different times, would have to find a contiguous region for every new one, and the gaps that finished sequences leave behind fragment memory, as in any heap allocator. A *paged* cache, like an operating system's virtual memory, divides the cache into fixed-size blocks of 16 positions. Each sequence holds a list of blocks, its *block table*, which need not be adjacent, and the scheduler takes blocks from a free list and returns them when the sequence finishes, so any free block can serve any sequence.

This chapter's scheduler reserves a sequence's blocks for its prompt and its longest permitted reply when it admits it. Production servers allocate each block only when the sequence reaches it, so that the only memory wasted is the unfilled part of each sequence's last block. Lab 4.8.2 makes that change and measures what it saves.

The block pool — code/aif/serving.py

```
class PagedCache:
    """Keys and values in fixed-size blocks; each sequence owns a list of them."""

    def __init__(self, model: GPT, blocks: int, block_size: int = 16):
        c = model.config
        p = next(model.parameters())
        shape = (c.n_layers, blocks, block_size, c.n_kv_heads, c.head_dim)
        self.k = torch.zeros(shape, device=p.device, dtype=p.dtype)
        self.v = torch.zeros(shape, device=p.device, dtype=p.dtype)
        self.block_size = block_size
        self.free = list(range(blocks))

    def blocks_for(self, tokens: int) -> int:
        return (tokens + self.block_size - 1) // self.block_size
```

Decoding a batch then computes each sequence's query, key and value for its new token together, writes each new key and value into its sequence's current block, gathers each sequence's keys and values through its block table, and runs attention with a mask that hides each sequence's unused positions:

One decode step for a batch of sequences of different lengths — code/aif/serving.py

```
@torch.no_grad()
def decode_step(
    model: GPT, cache: PagedCache, seqs: list[Sequence], tokens: torch.Tensor
) -> torch.Tensor:
    """One new token for every running sequence, in one batch: logits (N, vocab)."""
    dev = tokens.device
    pos = torch.tensor([s.length for s in seqs], device=dev)
    width = max(len(s.blocks) for s in seqs)
    table = torch.tensor([s.blocks + [0] * (width - len(s.blocks)) for s in seqs], device=dev)
    where = table[torch.arange(len(seqs), device=dev), pos // cache.block_size]
    slot = pos % cache.block_size
    span = width * cache.block_size
    visible = (torch.arange(span, device=dev)[None, :] <= pos[:, None])[:, None, None, :]
    cos, sin = model.cos[pos][:, None, None, :], model.sin[pos][:, None, None, :]
    x = model.embed(tokens)[:, None, :]
    for layer, block in enumerate(model.blocks):
        a = block.attn
        h = block.attn_norm(x)
        q = a.q(h).view(-1, 1, a.n_heads, a.head_dim).transpose(1, 2)
        k = a.k(h).view(-1, a.n_kv_heads, a.head_dim)
        v = a.v(h).view(-1, a.n_kv_heads, a.head_dim)
        q, k = apply_rope(q, cos, sin), apply_rope(k[:, :, None], cos, sin)[:, :, 0]
        cache.k[layer, where, slot] = k
        cache.v[layer, where, slot] = v
        keys = cache.k[layer][table].flatten(1, 2).transpose(1, 2)
        values = cache.v[layer][table].flatten(1, 2).transpose(1, 2)
```

```
        group = a.n_heads // a.n_kv_heads
        keys, values = keys.repeat_interleave(group, 1), values.repeat_interleave(group, 1)
        y = F.scaled_dot_product_attention(q, keys, values, attn_mask=visible)
        x = x + a.o(y.transpose(1, 2).reshape(len(seqs), 1, -1))
        x = x + block.mlp(block.mlp_norm(x))
    for s in seqs:
        s.length += 1
    return model.head(model.norm(x))[:, 0]
```

Production kernels read the blocks in place inside the attention kernel instead of gathering them into a padded tensor. The gather here costs memory traffic but keeps the code to a page.

Scheduling

Static batching collects a batch of requests, runs them until all have finished, and only then admits more, so a batch's short requests wait for its longest and new arrivals wait for the whole batch. *Continuous batching* admits a waiting request into the running batch as soon as a slot and enough blocks are free, so the batch stays full and nobody waits for anyone else's reply to finish.

The scheduler: admit, prefill, decode, retire — code/aif/serving.py

```
def run(
    model: GPT,
    cache: PagedCache,
    requests: list[Sequence],
    stop: int,
    max_batch: int,
    continuous: bool = True,
    clock=time.perf_counter,
) → float:
    """Serve requests in arrival order; without continuous, each batch runs to completion."""
    dev = next(model.parameters()).device
    pending = sorted(requests, key=lambda s: s.arrival)
    running = []
    start = clock()
    while pending or running:
        now = clock() - start
        can_admit = continuous or not running
        while can_admit and pending and pending[0].arrival ⩽ now and len(running) < max_batch:
            need = cache.blocks_for(len(pending[0].prompt) + pending[0].max_new)
            if need > len(cache.free):
                break
            seq = pending.pop(0)
            seq.blocks = [cache.free.pop() for _ in range(need)]
            logits = prefill(model, cache, seq)
            seq.out.append(int(logits.argmax()))
            seq.first = clock() - start
            running.append(seq)
        if not running:
            time.sleep(max(0.0, pending[0].arrival - (clock() - start)))
            continue
        tokens = torch.tensor([s.out[-1] for s in running], device=dev)
        logits = decode_step(model, cache, running, tokens)
        for s, t in zip(running, logits.argmax(-1).tolist()):
            s.out.append(t)
        for s in [s for s in running if s.out[-1] == stop or len(s.out) ⩾ s.max_new]:
            s.finish = clock() - start
            cache.free += s.blocks
            running.remove(s)
    return clock() - start
```

A load of requests arriving at random, and the measurements `code/u4/serve.py`

```
def benchmark(
    model, tok, ids, rate: float, max_batch: int, continuous: bool, max_new: int
) -> dict:
    arrivals = np.cumsum(np.random.default_rng(0).exponential(1 / rate, len(ids)))
    requests = [
        serving.Sequence(prompt=p, max_new=max_new, arrival=float(a))
        for p, a in zip(ids, arrivals)
    ]
    cache = serving.PagedCache(model, blocks=2048, block_size=16)
    makespan = serving.run(
        model, cache, requests, tok.special[chat.END], max_batch, continuous
    )
    ttft = np.array([r.first - r.arrival for r in requests])
    per_token = np.array([(r.finish - r.first) / max(1, len(r.out) - 1) for r in requests])
    generated = sum(len(r.out) for r in requests)
    return {
        "throughput": generated / makespan,
        "ttft": float(ttft.mean()),
        "ttft90": float(np.percentile(ttft, 90)),
        "itl": float(per_token.mean()),
        "makespan": makespan,
    }
```

48 chat requests at two per second, served three ways, and a check of batched decoding

```
$ uv run -m u4.serve
48 chat requests at 2 per second, up to 128 tokens each, SmolLM2-360M-Instruct in float16:
policy                      tokens/s  TTFT mean  TTFT p90  ms/token  makespan
one at a time                     46     40.34s    76.85s      21.4    116.7s
static batches of 16             158      2.07s     3.57s      32.2     33.9s
continuous batching, 16          176      0.06s     0.08s      33.9     30.5s
batched and one-at-a-time greedy decoding agree on 8/8 prompts in float16, 8/8 in float32
```

Serving one request at a time, the server generates 46 tokens per second, and each request's tokens come quickly, 21.4 ms apart. Requests arrive faster than it can serve them, however, so they queue: the mean wait for a first token is 40 seconds, and the last request finishes nearly two minutes after the first arrived. Static batches of 16 serve 3.4 times as many tokens per second and cut the mean wait to 2.1 seconds, at the cost of slower steps, 32.2 ms per token, since each step now processes up to sixteen sequences. Continuous batching raises throughput further, to 176 tokens per second, and cuts the mean wait to 0.06 seconds, the time of one prefill, because a request no longer waits for anyone else's reply to finish. Its tokens come 33.9 ms apart, about 60% slower than alone, because each step is shared. For almost every request the shorter wait repays that many times over.

The last line checks batching itself: eight prompts decoded together, and each decoded alone. A sequence decoded in a batch can choose a different greedy token from the same sequence decoded alone, because the batched kernels add the same numbers in a different order and two candidate tokens can be within rounding of each other. Here all eight prompts agree in both precisions, but nothing guarantees it, and near-ties are more frequent in float16, whose rounding is coarser. Users of a batched server should expect greedy outputs to be reproducible only up to such near-ties.

Speculative decoding

A large model's decode step costs the same whether it processes one token or several, as long as the memory traffic dominates, and *speculative decoding* uses this. A small *draft* model, cheap to run, proposes the next k tokens one at a time, and the large *target* model then processes all k in one pass and checks them. Each proposed token that matches what the target would have chosen is accepted. At the first mismatch, the target's own choice replaces the draft's and the rest are discarded. Every round therefore produces between one and $k + 1$ tokens, all of them exactly the target's greedy choices, for the price of one target pass and k draft passes.

Greedy speculative decoding, with both caches rolled back to the accepted tokens — code/aif/serving.py

```python
@torch.no_grad()
def speculative(
    target: GPT, draft: GPT, prompt: list[int], max_new: int, k: int, stop: int
) -> tuple[list[int], int]:
    """Greedy speculative decoding: the output is exactly the target's greedy output."""
    dev = next(target.parameters()).device
    limit = len(prompt) + max_new + k + 2
    t_cache = KVCache(target.config, 1, limit, dev, next(target.parameters()).dtype)
    d_cache = KVCache(draft.config, 1, limit, dev, next(draft.parameters()).dtype)
    target(torch.tensor([prompt[:-1]], device=dev), cache=t_cache)
    draft(torch.tensor([prompt[:-1]], device=dev), cache=d_cache)
    out, last, behind, rounds = [], prompt[-1], [], 0
    while len(out) < max_new and stop not in out:
        logits = draft(torch.tensor([behind + [last]], device=dev), cache=d_cache)[0, -1]
        proposal = []
        for i in range(k):
            proposal.append(int(logits.argmax()))
            if i < k - 1:
                logits = draft(torch.tensor([[proposal[-1]]], device=dev), cache=d_cache)[
                    0, -1
                ]
        start = t_cache.length
        checks = (
            target(torch.tensor([[last] + proposal], device=dev), cache=t_cache)[0]
            .argmax(-1)
            .tolist()
        )
        n = 0
        while n < k and proposal[n] == checks[n]:
            n += 1
        out += proposal[:n]
        last = checks[n]
        out.append(last)
        t_cache.length = start + 1 + n
        d_cache.length = start + 1 + min(n, k - 1)
        behind = proposal[k - 1 : k] if n == k else []
        rounds += 1
    out = out[:max_new]
    return out[: out.index(stop) + 1] if stop in out else out, rounds
```

SmolLM2-1.7B-Instruct drafted by SmolLM2-135M-Instruct, on eight prompts

```
speculative decoding in float32: SmolLM2-1.7B-Instruct drafted by SmolLM2-135M-Instruct
  target alone:  19.3 tokens/s
  k = 2:  26.5 tokens/s (1.37x), 80% of drafted tokens accepted, output identical
  k = 4:  25.3 tokens/s (1.31x), 67% of drafted tokens accepted, output identical
  k = 6:  24.4 tokens/s (1.26x), 59% of drafted tokens accepted, output identical
```

The target alone generates 19.3 tokens per second. Drafting two tokens at a time raises that to 26.5, 1.37 times as fast, with 80% of the drafted tokens accepted. Drafting four or six gains less, 1.31 and 1.26 times, because acceptance falls to 67% and 59% and every rejected token wasted a draft pass. The output is identical to the target's in every case, as the algorithm guarantees. The gain is modest because drafting has a cost. SmolLM2-135M has a thirteenth of the target's parameters, but a decode step of a small model on a GPU costs more than its share of the arithmetic, as Chapter 2.7 found for the course's model. Speculative decoding pays most when the draft is much cheaper per step than the target, and when the two agree often.

With sampling instead of greedy choice, a rejection-sampling rule accepts each draft token with probability $\min\left(1, p_{\text{target}}/p_{\text{draft}}\right)$ and resamples from the leftover probability on rejection, which provably gives exactly the target's sampling distribution. Lab 4.8.4 implements it.

A streaming endpoint

The scheduler runs as an asynchronous task; each HTTP request adds a sequence with a queue, and the handler streams tokens from the queue to the client as server-sent events as the scheduler produces them:

The scheduler as an asyncio task, feeding each request's queue — code/u4/serve.py

```python
async def scheduler(model, tok, queue, max_batch: int = 16) -> None:
    cache = serving.PagedCache(model, blocks=2048)
    stop, running = tok.special[chat.END], []
    dev = next(model.parameters()).device
    while True:
        while not queue.empty() and len(running) < max_batch:
            seq = queue.get_nowait()
            seq.blocks = [
                cache.free.pop()
                for _ in range(cache.blocks_for(len(seq.prompt) + seq.max_new))
            ]
            seq.out.append(int(serving.prefill(model, cache, seq).argmax()))
            seq.sink.put_nowait(seq.out[-1])
            running.append(seq)
        if not running:
            seq = await queue.get()
            await queue.put(seq)
            continue
        logits = serving.decode_step(
            model, cache, running, torch.tensor([s.out[-1] for s in running], device=dev)
        )
        for s, t in zip(running, logits.argmax(-1).tolist()):
            s.out.append(t)
            if t != stop:
                s.sink.put_nowait(t)
        for s in [s for s in running if s.out[-1] == stop or len(s.out) >= s.max_new]:
            s.sink.put_nowait(None)
            cache.free += s.blocks
            running.remove(s)
        await asyncio.sleep(0)
```

```
uv run -m u4.serve --http 8000
curl -N -d '{"prompt": "Write a haiku about the sea."}' http://127.0.0.1:8000/generate
```

KEY IDEA

Decoding is memory-bound, so a server's efficiency comes from reading the weights once for many tokens: continuous batching fills each step with many sequences, a paged cache lets them share memory without fragmenting it, and speculative decoding lets one pass of a large model accept several cheaply drafted tokens.

IN PRACTICE

vLLM introduced the paged KV cache, as PagedAttention, and combines it with continuous batching, speculative decoding, prefix caching and quantised kernels; SGLang and TensorRT-LLM are its main alternatives, and `llama.cpp` has a server of its own. All expose an HTTP interface compatible with OpenAI's API, which streams tokens much as this chapter's endpoint does.

Reading

- Woosuk Kwon and others, "Efficient Memory Management for Large Language Model Serving with PagedAttention", SOSP 2023.
- Gyeong-In Yu and others, "Orca: A Distributed Serving System for Transformer-Based Generative Models", OSDI 2022: iteration-level scheduling.
- Yaniv Leviathan, Matan Kalman and Yossi Matias, "Fast Inference from Transformers via Speculative Decoding", ICML 2023.
- Charlie Chen and others, "Accelerating Large Language Model Decoding with Speculative Sampling", 2023.

Labs

LAB 4.8.1 Load and latency — 1.5 hours

Goal. Plot latency and throughput against the arrival rate.

Code. `code/u4/serve.py`

1. Run the continuous-batching benchmark at arrival rates of 0.5, 1, 2, 4 and 8 requests per second.
2. Plot median and 90th-percentile time to first token, and throughput, against the rate.

DONE WHEN

- [ ] Your plot has five rates.
- [ ] You can name the rate at which latency starts to climb steeply, and relate it to the throughput at that point.

EVIDENCE FOR THE NOTEBOOK

- The plot and the analysis.

LAB 4.8.2 Blocks on demand — 1.5 hours

Goal. Allocate cache memory as sequences grow.

Code. `code/aif/serving.py`

1. Change the scheduler to give a sequence blocks for its prompt when it is admitted, and one more block whenever its length reaches the end of its last block.
2. Record, at every step of a benchmark, the blocks allocated and the positions actually filled, with reservation and on demand, for block sizes of 4, 16 and 64.

DONE WHEN

- [ ] Your table gives the average fraction of allocated cache in use for each block size under both policies.
- [ ] You can say what an on-demand scheduler must do when a sequence needs a block and none is free, and what each option costs.

EVIDENCE FOR THE NOTEBOOK

- The code and the table.

LAB 4.8.3 Prefix caching — 2 hours

Goal. Share the cache of a common system prompt.

Code. `code/aif/serving.py`

1. Give every request the same long system prompt. Compute its blocks once and let every sequence's block table start with them, with blocks counted so that shared ones are not freed while in use.
2. Measure time to first token with and without sharing.

DONE WHEN

- [ ] Outputs are unchanged by sharing.
- [ ] Your measurements show the saving, and you can relate it to the prompt's length.

EVIDENCE FOR THE NOTEBOOK

- The code and the measurements.

LAB 4.8.4 Speculative sampling — 2 hours

Goal. Apply speculative decoding to sampling and keep the target's distribution.

Code. `code/aif/serving.py`

1. Implement the rejection rule: accept each draft token with probability $\min\left(1, p_{\text{target}}/p_{\text{draft}}\right)$, and on rejection sample from the normalised positive part of $p_{\text{target}} - p_{\text{draft}}$.
2. Check the output distribution on a toy vocabulary by comparing token frequencies from 10,000 runs with the target's own sampling, then measure the speed-up on SmolLM2 at temperature 1.

DONE WHEN

- [ ] Your frequencies agree with the target's within sampling error.
- [ ] You have the speed-up and acceptance rate at temperature 1.

EVIDENCE FOR THE NOTEBOOK

- The test, the speed-up and the acceptance rate.

Problem set

1. A decode step of SmolLM2-360M in float16 reads 720 MB of weights. On a machine with 150 GB/s, what is the step's time for a batch of 1, and up to what batch size does it stay about the same on a GPU with 6 TFLOP/s?
2. Static batching runs batches of 16; requests' replies are 20 to 128 tokens long. What fraction of the batch's slots is idle on average, if lengths are uniform?
3. With a draft that agrees with the target 70% of the time per token and $k = 4$, what is the expected number of tokens per round? If a draft pass costs a tenth of a target pass, what is the speed-up?
4. Why must both caches be rolled back after a rejection, and what would go wrong if the draft's were not?
5. Continuous batching admits long prompts whose prefill stalls everyone's decoding. How could a scheduler limit that, and what does it cost?

COMPLETION CHECKLIST

- [ ] I can explain why batching raises decode throughput and measure throughput and latency under load.
- [ ] I can implement a paged KV cache and batched decoding of sequences of different lengths.
- [ ] I can implement continuous batching and compare it with static batching.
- [ ] I can implement speculative decoding and show that it preserves the target's output.
- [ ] Lab 4.8.1–4.8.4 done-when criteria all hold, with evidence filed.

UNIT 4

Sign-off

A unit is complete when every chapter's completion checklist is ticked, its evidence is in your notebook and repository, and you can do each review task below without notes. Date each line as you finish it. If someone who trains models for a living will review your work, ask them to sign the last line.

	Chapter	Checklist ticked	Evidence filed	Date
4.1	The machine	☐	☐	
4.2	Profiling and memory	☐	☐	
4.3	Numerics	☐	☐	
4.4	Kernels	☐	☐	
4.5	Parallel training	☐	☐	
4.6	An inference engine in C++	☐	☐	
4.7	Quantisation	☐	☐	
4.8	Serving	☐	☐	

UNIT REVIEW

Each task takes an hour or less if the unit has done its work. Do them aloud or in writing, from memory, then check against your notebook and code.

- ☐ Place matrix multiplication, attention, a norm and the cross-entropy on your machine's roofline, from your measurements, and say what limits each.
- ☐ Write down the memory needed to train a 7-billion-parameter model with AdamW in mixed precision, before activations, and say which parallelism you would use to fit it on eight 80 GB GPUs.
- ☐ Explain FlashAttention's online softmax, and why it reads and writes less memory than materialised attention.
- ☐ Explain why decoding one token at a time is memory-bound, and how quantisation, batching and speculative decoding each attack that bound.
- ☐ Describe how continuous batching and a paged KV cache raise a server's throughput, with your Chapter 4.8 measurements.

Signed: student _____ Date _____

Reviewed by (optional) _____ Date _____